BUBAS

BUBAS

Orchestration for people who own the rules

Peter Verhas

Every program, transcript and compiler message in this book is produced by the build that produced the book. Nothing here was typed from memory.

Contents

30. Failing well
31. Where programs come from
32. Programs written by a model
33. Running someone else's rules

PART 1

The Language

CHAPTER 1

Why a small language

The problem before the solution: a rule nobody accountable can read, approved by somebody who can only check that it looks reasonable. Why this is a question of authority rather than competence, and why it does not go away when the tooling gets better. Ends with the alternative — do not constrain a large language, supply a small one — and what that costs.

The review that does not happen

A pull request arrives. Two hundred lines of Java implementing the new expense policy: per-category caps, a receipt threshold, an exception for field engineers that nobody can quite explain, and an escalation path that depends on the claimant's grade. It compiles. The tests pass.

Somebody now has to approve it.

The person who owns that policy works in finance. She wrote the rules, she argued them through with the works council, and she is the one who will be asked to explain them when an auditor comes calling. She cannot read Java.

The person who can read Java is a backend engineer. He has no idea whether field engineers really are exempt from the receipt threshold, or whether the escalation is supposed to trigger at the claimant's grade or their manager's. He will check that the code looks reasonable, because that is the only thing he is equipped to check.

So the review that happens is not the review that matters. One person can read the artefact and cannot judge it; the other can judge it and cannot read it. The approval goes through, and what has actually been verified is that the code is not obviously badly written.

This is not a story about a badly run team. Every part of it is normal. The engineer is doing his job properly. The finance manager is not being lazy — she has been handed a document in a language she has no reason to know. The gap between them is structural, and it is where this book starts.

Competence is not the issue

Most complaints about code that implements business rules — and especially about generated code — are complaints about competence. It gets an edge case wrong. It handles the happy path and falls over in production. It hallucinates a method that does not exist.

Those complaints expire. Tooling improves, models improve, and any position resting on today's error rate has a shelf life. People who staked one out a few years ago have mostly had to retreat from it, quietly.

The durable question is a different one. It is not how well the rule is written. It is **what the thing it is written in is permitted to say.**

Put it as a question about a town. Would you rather live in one where nobody carries a gun, or in one where everybody does and your safety rests on each of them being sane and well-intentioned? The second town can be perfectly peaceful for years. It is not peaceful for the same reason.

A perfectly competent engineer, writing Java, can still reach the file system. Not because he intends to, and not because he is careless, but because that sentence is available in the language he was asked to write. Competence and authority are separate axes, and improving one does nothing to the other. A flawless implementation of your expense policy, written in a general-purpose language, still sits inside a program that could open a socket.

That matters less when a trusted colleague writes it and more when the author is a contractor, a distant team, or a model. But the structural fact is the same in all four cases, and it is the fact this book is about.

What "write it in Java" authorises

Consider what you actually grant when you ask for an expense rule in Java.

You grant file system access. Network sockets. Reflection. Thread creation. Process execution. Every class on the classpath, including the ones that talk to your database, your payment provider and your secrets manager. You grant the loading of new code at run time.

Nobody intends to grant any of this. It arrives with the language, the way a key to the building also opens every cupboard in it. The task needed perhaps six operations — look at a claim, total it, check a receipt, compare against a limit, record a decision, notify somebody — and the language handed over contains everything Java contains.

That gap, between the authority the task requires and the authority the language confers, is the whole of the problem. It exists whether or not the author is trustworthy. It exists whether or not anyone ever acts on it. It is simply very large, and it is invisible in the diff.

The usual answer is a deny-list

The standard responses all have the same shape.

Review the code. Run static analysis and flag dangerous calls. Forbid certain imports. Run it in a sandbox with a restricted policy. Tell the author, in the ticket, not to touch the file system.

Every one of these asks you to enumerate what must not happen, across a space of things that can happen which is effectively unbounded. You have to think of process execution. Then of reflection reaching process execution. Then of the dependency that shells out on your behalf. Then of the next one, which you have not thought of yet, which is the entire difficulty.

Security settled this argument a long time ago and reached an answer: allow-lists beat deny-lists, because the allow-list is finite and you wrote it. Somehow, when the subject is business logic, we reach for the deny-list again.

Supply a small language instead

The alternative is to stop constraining a powerful language and instead supply a small one.

Give the author a vocabulary containing exactly the operations the domain has, and nothing else. Not a restricted Java — a different, much smaller language, whose entire vocabulary is a list your team wrote in advance, in Java, on purpose.

Here is an expense rule in such a language:

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS BOOLEAN
    DECLARE total DECIMAL

    total = TOTAL_OF(claim)

    IF total > limit THEN
        REJECT claim, "over the " + limit + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

`TOTAL_OF`, `APPROVE` and `REJECT` are not part of BUBAS. BUBAS has no such words. They are Java classes somebody on your team decided to expose, and in a different application the same language would contain a completely different set. `Report` is a Java object the program can hold and pass and never look inside — there is no `claim.employee.manager.email` here, only the operations the domain chose to have.

You read that program. You probably read it without deciding to. More usefully: you can tell whether it is *right*. Should a claim over the limit be refused outright, or sent to a manager? Should the claimant be shown the limit in the rejection? You may have opinions about those, and having opinions about those is exactly what the finance manager was unable to do with two hundred lines of Java.

Two things change, and the second matters more

The obvious change is that dangerous programs are no longer forbidden — they are **inexpressible**. If a program says `DELETE_ALL_CLAIMS`, nothing rejects it on policy grounds. The name means nothing. The program does not compile, for the same reason a typo does not compile. There is no deny-list because there is nothing to deny.

The less obvious change is that the review moves to the person who owns the rule. She can read the program above, and she can be held to it. That is the review that was missing at the start of this chapter, and no amount of static

analysis over generated Java produces it.

There is a third thing, quieter than either. With no data structures, one scope, no null, and a compiler that refuses to run a program which reads a variable before it has been set, entire families of subtle wrongness have nowhere to live. Not caught — absent. Chapter 8 is about what that feels like from the inside.

What it costs

Any argument that only lists advantages should be distrusted, so here are the bills.

You have to design the vocabulary. Somebody sits down and decides that this domain has `TOTAL_OF` and `HAS_RECEIPT` and not forty other things. That is real work, done before the first rule is written, by someone who understands the domain. If nobody on your team can write that list, this approach will not help you — it will only show you that the list does not exist. Worth finding out, but not a pleasant morning. Part 3 is largely about doing this well.

Complex algorithms stay in Java. Business rules are algorithms too, and they belong in the small language; that is the point. But route optimisation, a scoring model, anything with real computational substance belongs behind an operation the small language calls. The usual signal is that you want to build up a data structure, or want a helper you can call from three places. Both mean you have wandered out of business logic and should walk back. Chapter 9 gives this its own treatment.

The boundary bounds naming, not doing. This is the limit people miss, and overstating it is how the whole idea gets dismissed. An operation you expose can do anything Java can do. `RUN_SHELL_COMMAND` is a perfectly registrable operation. The vocabulary is only as narrow as the operations you chose, and choosing them badly gets you exactly the exposure you were avoiding.

Resource limits are shallower than they look. A run can be given a step budget and a cap on what a single array may allocate, and both are one line each on the interpreter — chapter 28 shows them. What is not there is a deadline: an operation you registered that blocks for an hour blocks for an hour, because the budget counts what the *program* does and that call is one step. Untrusted input

still needs the containment any untrusted workload needs. Chapter 33 is honest about where the line now falls.

What you get is narrower than "safe" and more useful than it sounds: the set of things a program can name is finite, written down, and reviewable by a human before anything is written at all.

When not to do this

If the thing is genuinely computational, write Java.

If it is a one-off that will be deleted next week, use whatever is nearest and do not build a vocabulary for something with a life expectancy of days.

If your rules change so fast that the vocabulary would be obsolete before it settled, the overhead will not pay for itself — though in practice, domains whose *operations* churn that fast are rarer than domains whose *thresholds* churn, and thresholds are cheap.

And if your business logic is already read, understood and approved by the people accountable for it being right, you may not have the problem this solves. Plenty of teams do not.

How to read this book

The book is in three parts.

Part 1 is the language: how to read a BUBAS program and how to write one. It contains no Java at all, and it is written for whoever owns the rules as much as for the engineer who will embed the language. Both need it, for the same reason — you cannot design a language you have not learned to use.

Part 2 is testing. BUBAS programs are tested in BUBAS, so that the person who reviews the rule can also read the check on it. Also no Java.

Part 3 is embedding: defining a vocabulary, wiring it into an application, and running it in production. This is where Java lives, and Parts 1 and 2 point forward to it constantly, because a rule writer who wants an operation that does not exist yet needs somebody to go and build it.

One worked application runs through all three parts: approving employee expense claims. It begins as a total and a limit, and grows into something with receipts, per-category caps, escalation to a manager, and an anomaly score from a model. Nothing is ever rewritten to make a later chapter work. Each chapter adds.

Expense approval is used because you can audit it. Shown a rule that pays a €340 dinner for one person with nothing attached, you know something is wrong without being told. That is the experience this book is trying to give you: not the claim that experts can review these programs, but a page on which you do it.

The next chapter is a complete program, start to finish.

CHAPTER 2

A first rule

A complete program, start to finish: a claim comes in, a total is worked out, a decision is made and recorded. Program structure, parameters, `DECLARE`*,* `RETURN`*, and what running it looks like. By the end of this chapter you can read a BUBAS program, which is most of what this book is for.*

The whole thing

Here is a complete BUBAS program. Not an excerpt, not a simplified version — this is the entire file, and it compiles and runs exactly as printed:

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS BOOLEAN
    DECLARE total DECIMAL

    total = TOTAL_OF(claim)

    IF total > limit THEN
        REJECT claim, "over the " + limit + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

Thirteen lines. It decides whether an expense claim can be paid without anyone looking at it.

The rest of this chapter takes it apart. If you read the program and understood it, that is the correct reaction and you should not be suspicious of it — the language is small on purpose, and a program that needed explaining would be a failure of the design rather than a sign of depth.

The first line

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS BOOLEAN
```

Every program says what it is called, what it needs, and what it answers.

`ApproveExpense` is the name. It matters when the surrounding application goes looking for a rule to run; within the file, nothing refers to it.

`claim Report` and `limit DECIMAL` are **parameters** — the things the program is given. Each is a name followed by its type, in that order, because you are naming a thing and then saying what kind of thing it is. The claim to decide about, and the limit to decide against.

`RETURNS BOOLEAN` says the program answers true or false. A program need not answer anything; if it only records decisions and notifies people, leave `RETURNS` off entirely. This one answers, because whatever called it wants to know whether to pay.

Parameters are the whole reason a program is worth compiling once and running many times. The same compiled `ApproveExpense` decides every claim that arrives all day, and it decides them differently because it is given different things:

```
ApproveExpense(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 128.40
    => TRUE

ApproveExpense(claim = report 1 (Alice), limit = 100.00)
    rejected report 1 (Alice) — over the 100.00 limit
    => FALSE
```

One claim, two limits, two answers, and nothing recompiled in between.

Saying what you will need

```
DECLARE total DECIMAL
```

Every variable is announced before it is used, with its type. There is no way to invent a variable by assigning to it, and no way to leave the type to be worked out later.

That is more ceremony than a scripting language would ask for, and it is deliberate. A rule that mentions `totl` on line nine is a rule with a bug that nobody will find by reading, and there is no good reason for a language in this position to allow it. Declare it, or you cannot use it.

`DECIMAL` is the type for money. There is a separate type for whole numbers, and later chapters use both. What matters here is that `DECIMAL` means what an accountant means: 0.10 plus 0.20 is 0.30, exactly, and never 0.30000000000000004. Chapter 3 goes into why that is worth a type of its own.

Asking a question

```
total = TOTAL_OF(claim)
```

`claim` needed no working out — it arrived as a parameter, because the application already had it in hand when it decided to run this rule. `total` is different. It has to be worked out, and working it out means asking something that knows how.

`TOTAL_OF` is an **operation**. It is not part of BUBAS: someone on the team that set up this language decided that "what does this claim come to" was a question worth being able to ask, and made it askable. In a different application it would not exist, and something else would.

Operations that answer a question are written with brackets and used inside an expression, the way you would expect. Chapter 4 deals with the other kind, which does not answer and is written differently.

`Report` is a type this language has and Java does not care about: the claim itself. A program can hold one, pass it to an operation, and keep it in a variable. What it cannot do is look inside. There is no way to write `claim.employee` or `claim.lines[0].amount`, because there is no syntax in the language for reaching into anything at all. Chapter 5 is about what that is like to live with, and why it is the single most important decision in the design.

Deciding

```
IF total > limit THEN
    REJECT claim, "over the " + limit + " limit"
    RETURN FALSE
END IF
```

A test, some statements, and `END IF`. The block is closed explicitly rather than by indentation, so a rule cannot change meaning because someone's editor

reformatted it.

`REJECT` is the other kind of operation. It does not answer a question — it does something, and there is nothing to assign. So it is written as its own statement, with no brackets: the name, then whatever it needs, separated by commas. Compare `TOTAL_OF(claim)`, which you use for its answer, with `REJECT claim, "..."`, which you use for its effect. You can tell them apart on sight, which is the point.

The reason string is built with `+`. There is no formatting language, no placeholders, no percent signs; a number joined to text becomes text. `"over the " + limit + " limit"` produces `over the 200.00 limit`, and that is as complicated as string building gets.

`RETURN FALSE` answers immediately and stops. Nothing after it runs.

Answering

```
APPROVE claim
RETURN TRUE
```

If the claim survived the test, it is approved and the program says so.

`APPROVE` takes one thing and needs no comma. `RETURN TRUE` gives the answer that `RETURNS BOOLEAN` promised on line one. A program that says it returns a `BOOLEAN` must actually return one on every path out — the compiler checks, and a rule that could fall off the end without answering does not compile.

The last line

`END` closes the program. The full stop after it is optional — `END` alone is accepted — and it is there as a tribute to Niklaus Wirth, whose Pascal ended every program that way. Wirth spent his last years in Winterthur, about seven kilometres from where this language was written.

Write it or leave it off. The one habit worth keeping is consistency within a codebase, since a reader who sees both starts wondering which one means something.

Running it

Two claims, both against a limit of 200.00:

```
ApproveExpense(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 128.40
    => TRUE

ApproveExpense(claim = report 2 (Bob), limit = 200.00)
    rejected report 2 (Bob) — over the 200.00 limit
    => FALSE
```

Alice's taxi, lunch and hotel come to 128.40 and are approved. Bob's conference fee and dinner come to 1230.00, and are not.

The indented lines are what `APPROVE` and `REJECT` recorded. What that means in practice — a row in a table, a message on a queue, an email to the claimant — is entirely up to the application, and the rule neither knows nor can find out. It says what should happen. Something else makes it happen.

What it cannot say

It is worth noticing what is absent, because it is absent by construction rather than by convention.

There is no way to open a file, reach the network, start a thread, or call anything that was not put in front of the program deliberately. Not because a checker forbids it — because there are no words for it. If this program said `SEND_TO_AUDITORS claim`, it would not compile, for exactly the same reason that `SNED_TO_AUDITORS` would not: the name means nothing.

That is a smaller guarantee than it sounds and a more useful one than it sounds, and chapter 1 argued it at length. Here it is enough to notice that the guarantee is visible in the program: you can see everything this rule is able to do, because it is all on the page.

What you can now read

You can read a BUBAS program. That is not a small claim to have earned in thirteen lines.

What you cannot yet do is write one from nothing, or predict what the compiler will refuse. The next chapters fill that in — values and types, the two kinds of operation, opaque types, then decisions and loops — and by chapter 8 you will know what will not compile and why the list is shorter than you expect.

CHAPTER 3

Values

The four kinds of value a program can hold — whole numbers, decimals, text, true and false — and why money is always `DECIMAL` *and never a floating-point approximation of itself. There is no null: a value that has not been worked out yet cannot be read at all, a rule with consequences taken up in chapter 8.*

What a program can hold

Four kinds of value, and that is the whole list:

Written as	Called	For
`42`, `0`, `-10`	`INTEGER`	whole numbers — counts, days, positions
`3.14`, `0.5`, `1000.0`	`DECIMAL`	money, rates, anything with a fractional part
`"over the limit"`	`STRING`	text
`TRUE`, `FALSE`	`BOOLEAN`	yes and no

There is a fifth kind, and it behaves differently enough to get its own chapter: values from the domain itself, like the `Report` in the last chapter. A program can hold one and pass it around but cannot look inside it or write one down. Chapter 5 is about those.

Everything else you might expect is missing. There are no dates, no lists, no records, no maps, no objects with fields. If your rule needs to know whether a claim was filed within thirty days, that is a question for an operation, not something you assemble out of primitives. This is a language for *deciding*, and the things it can hold are the things decisions are made of.

Whole numbers

`INTEGER` is a 64-bit signed whole number, so it runs to a little over nine quintillion in either direction. For counting days, line items or people, it will not

run out.

Three behaviours are worth knowing before they surprise you.

Division throws away the remainder. `7 / 2` is `3`, not `3.5`. `-7 / 2` is `-3` — it truncates toward zero rather than rounding down. If you want the fractional part, one side has to be a `DECIMAL`.

`MOD` **gives the remainder**, and takes its sign from the left-hand side: `-7 MOD 2` is `-1`.

Overflow is an error, not a wraparound. A calculation that runs past the end of the range stops the program rather than quietly continuing with a negative number. This is the opposite of what most languages do, and it is the right choice for a language deciding what to pay: a rule that silently produces nonsense is worse than one that stops.

Dividing by zero is also an error rather than a special value.

Money

`DECIMAL` is the type for anything you would put in a currency column, and it is the reason this chapter exists.

Most programming languages, asked to add 0.10 and 0.20, will tell you the answer is 0.30000000000000004. That is not a bug; it is what happens when you store base-ten fractions in base two, and every experienced programmer has learned to work around it. BUBAS does not have that problem to work around, because `DECIMAL` is not a floating-point number. It stores the digits you wrote.

Here is a claim for ten cents and twenty cents, run through a per-diem rule:

```
ApprovePerDiem(claim = report 6 (Nadia), dailyCap = 20.00, days = 1)
    approved report 6 (Nadia) for 0.30
    => TRUE

ApprovePerDiem(claim = report 7 (Omar), dailyCap = 20.00, days = 1)
    approved report 7 (Omar) for 10.50
    => TRUE

ApprovePerDiem(claim = report 8 (Priya), dailyCap = 30.00, days = 3)
    rejected report 8 (Priya) — averages
    ■ 33.333333333333333333333333333333333 a day over 3 days, cap is
    ■ 30.00
    => FALSE
```

Three things are on display there.

Addition, subtraction and multiplication are exact. Ten cents plus twenty cents is thirty cents, and there is no version of this language in which it is not.

Scale is preserved. Omar's bus fare renders as `10.50`, not `10.5`. The trailing zero is information — it is the difference between a price and a rounded price — and the language keeps it.

Division is the exception, and it shows. Priya's hundred euros over three days comes out as `33.333333333333333333333333333333333`: thirty-four significant digits, which is as far as the default settings go. Division cannot be exact — a third has no finite decimal expansion — so somewhere a line has to be drawn, and the application draws it. An application that wants two decimal places and banker's rounding says so when it builds the language, and every division in every rule follows suit — in testing exactly as in production.

Arithmetic on figures written in the rule is worked out while the rule is compiled. A cap of `50.00 * 3` is the number 150.00 in the compiled program, and nothing adds it up again on each run. This is invisible while it is only saving work. It stops being invisible when the answer decides something: a test on a figure the rule worked out for itself has an answer before the rule runs, and chapter 8 explains why that is refused rather than allowed.

The practical consequence for a rule writer: **do not divide unless you mean to**. Comparing a total against a cap is exact and always will be. Working out an average and comparing that is a calculation whose last digits depend on a setting somewhere else, which is fine as long as you are not deciding a marginal case on

the thirtieth decimal place.

An `INTEGER` used where a `DECIMAL` is wanted is widened automatically. The rule above is declared with one of each and divides them without ceremony:

```
PROGRAM ApprovePerDiem(claim Report, dailyCap DECIMAL, days INTEGER)
■ RETURNS BOOLEAN
    DECLARE total DECIMAL
    DECLARE perDay DECIMAL

    total = TOTAL_OF(claim)
    perDay = total / days
```

`days` is an `INTEGER`, `total` is a `DECIMAL`, and `perDay` is a `DECIMAL` because that is what the division produces. The widening only goes one way: a `DECIMAL` where an `INTEGER` is wanted is an error, because narrowing would have to throw something away and the language will not choose what.

Text

`STRING` is text in double quotes, with `\n`, `\t`, `\r`, `\\` and `\"` available as escapes.

Text is joined with `+`, and there is exactly one rule to remember: **the left-hand side must already be text**. `"Count: " + 42` gives `Count: 42`. Writing `42 + " items"` is an error, because the left side is a number and BUBAS will not silently decide you meant text. Start with `""` if you need to lead with a number.

The rule has one consequence that catches people. Since `+` groups from the left, `"n=" + 1 + 2` is `n=12`, not `n=3`. The first `+` produces text, so the second one joins to it. If you want the sum, bracket it.

There is no formatting language — no placeholders, no percent signs, no format strings to learn. Numbers render the way you would write them: whole numbers in plain digits, decimals in plain notation with their scale kept and never in scientific form, and `TRUE` or `FALSE` exactly as you would type them.

A domain value cannot be turned into text by the language. BUBAS has no idea what a `Report` looks like, so `"claim " + claim` is an error. What the domain can do is provide an operation for it — something that answers a `STRING` — and then the words are the domain's own rather than a rendering BUBAS

invented. If your vocabulary has no such operation and a message needs to name the claimant, that is an operation to ask for.

True and false

`BOOLEAN` is `TRUE` or `FALSE`, written in capitals, and there is nothing else in the type. No truthy numbers, no empty string counting as false, no null pretending to be false. A condition is either a `BOOLEAN` or it does not compile.

Two booleans can be compared with `=` and `<>` and nothing else — asking whether `TRUE` is greater than `FALSE` is not a question the language will accept, because it is not a question.

Comparing things

All six comparisons — `=`, `<>`, `<`, `>`, `<=`, `>=` — work on numbers, and on text, where they order it character by character. Note that equality is written `=`; there is no `==` in this language, and `<>` rather than `!=` is inequality.

Numbers of different kinds compare fine; the `INTEGER` is widened to `DECIMAL` first. And `DECIMAL` comparison is by *value*, not by how it was written, so `2.0 = 2.00` is `TRUE`. That is worth knowing precisely because scale is otherwise preserved so carefully: the language keeps the trailing zero when showing you the number, and ignores it when comparing.

Domain values cannot be compared at all — not even for equality. Chapter 5 explains why that follows from what they are, and what to do when you genuinely need to know whether two claims are the same one.

The absence that is not there

There is no null in BUBAS. No `NULL` literal, no test for it, nothing that produces it.

This is not a claim that nothing can ever be missing. The world is full of missing things: a claim number that matches nothing, an approver who has left. What is absent is the *language-level* value that stands for all of them at once and behaves like a valid value right up until it does not.

A rule that has to handle absence asks a question about it, through an operation built for the purpose — `IF CLAIM_WAS_FOUND(claim) THEN`, or whatever that domain calls it. It is an ordinary operation returning an ordinary `BOOLEAN`, provided by whoever built the vocabulary because this domain needed it. The absence is named, in domain terms, at the point where it matters — rather than being a universal shadow value that every operation must defend against.

There is a second half to this, and it is the more useful one. A variable that has been declared but not yet worked out cannot be read *at all*. Not "reads as null", not "reads as zero" — the program does not compile. Chapter 8 is about that rule and the family of bugs it eliminates.

Names and types

One rule about names belongs here rather than later, because it bites when you first write a declaration.

Every name in a program — variables, operations, and the domain types — shares a single namespace, and names are unique **case-insensitively**. `Report` is a type in this language, so `report` is taken, and so are `REPORT` and `rEpOrT`. That is why the last chapter's program called its variable `claim`.

Once a name is declared, references to it must match it exactly, character for character. So `userId` and `UserID` cannot both exist, and having declared `userId` you cannot then write `UserId`. One rule rules out both the lookalike pair and the capitalisation typo.

Naming a variable after its type is the first thing most people reach for, so expect to meet this rule early. The compiler says plainly what happened.

What is coming

You now know what a BUBAS program can hold. The next chapter is about the two kinds of operation — the ones that answer questions and the ones that do things — and after that, the domain values that this chapter kept deferring.

CHAPTER 4

Asking and telling

Two kinds of operation, distinguished on sight. A function is asked a question inside an expression and answers with a value; a command is told to do something and stands alone on its line. Why the distinction is worth making, and how to tell which one you are looking at.

Two things a program does

A rule does two kinds of thing. It finds things out, and it makes things happen.

Finding out what a claim comes to, whether a receipt is attached, how many days the trip lasted — those are questions, and a question has an answer you then do something with. Approving the claim, refusing it, sending it to a manager, recording a note — those are not questions. There is nothing to do with the result, because the point was the doing.

BUBAS keeps these apart, and writes them differently, so you can tell at a glance which one you are looking at.

Asking

An operation that answers a question is written with brackets and used inside an expression:

```
total = TOTAL_OF(claim)
```

The brackets are not optional here, even when there is nothing to put in them. An asking operation with no arguments is still written `SOMETHING()`, because the brackets are what say *this is a call* rather than *this is a variable*.

The answer has to go somewhere. Usually into a variable, as above, but anywhere a value is welcome will do — inside a comparison, as an argument to something else, as the thing you return.

Telling

An operation that makes something happen stands alone as a whole line:

```
APPROVE claim
RETURN TRUE
```

`APPROVE claim` is one such line. It takes one thing and needs no punctuation at all. Where a telling operation takes more than one thing, they are separated by commas — `REJECT claim, "over the limit"` — and still no brackets.

There is nothing to assign, because there is no answer. `APPROVE` does not report whether it worked; if it cannot do its job it stops the program, and if it can, the program carries on.

The brackets are optional when telling

Some telling operations accept brackets as well. Both of these lines call the same operation, in the same program:

```
NOTE "checked against a limit of " + limit
```

```
NOTE("over by " + (total - limit))
```

Running it shows both notes coming out in order, alongside the decision:

```
ApproveWithNote(claim = report 1 (Alice), limit = 200.00)
    checked against a limit of 200.00
    approved report 1 (Alice) for 128.40
    => TRUE

ApproveWithNote(claim = report 2 (Bob), limit = 200.00)
    checked against a limit of 200.00
    over by 1030.00
    rejected report 2 (Bob) — over the 200.00 limit
    => FALSE
```

Write whichever reads better. `NOTE "checked the limit"` reads like an instruction, which is what it is; `NOTE("over by " + (total - limit))` puts brackets round a longer expression and is easier to scan for it. Nothing depends on the choice.

The reason it is allowed only in this direction is worth a sentence, and it is not ambiguity. BUBAS has one namespace, so `TOTAL_OF` is either an operation or a variable and never both; the compiler always knows which. The brackets are for the reader. Inside an expression, where a name could as easily be a value somebody stored earlier, `TOTAL_OF(claim)` says *this is being worked out here* at a glance. A telling operation is a whole line of its own, so a reader is in no doubt about it either way, and the language does not insist.

An answer cannot be thrown away

The one thing you cannot do is ask a question and ignore what comes back:

```
line 4: TOTAL_OF returns a value, so it cannot stand alone as a
■ statement; a result would be discarded silently
        TOTAL_OF claim
```

This is a small rule with a real purpose. In most languages, calling something for its answer and then discarding it is legal, and it is a classic way for a bug to hide: the call looks like it did something, and it did nothing anybody kept. Here, if an operation answers, its answer has to go somewhere, so a line that appears to do work always does.

The mirror image is also enforced. A telling operation cannot be used inside an expression, because it has nothing to contribute to one.

Almost every line is a telling

Here is the part that surprises people who have written other languages.

```
DECLARE total DECIMAL
```

`DECLARE` is not syntax. It is a telling operation like any other, supplied by a standard module that almost every BUBAS language installs, and an application that wanted to spell declarations differently would simply not install it. The same is true of assignment: `total = TOTAL_OF(claim)` is an operation whose shape happens to include an equals sign in the middle.

Nothing in the language privileges either of them, and nothing in the runtime treats them specially. They are shipped because a language without a way to

declare and assign is unusable, not because they are part of what BUBAS *is*.

That has a practical consequence. Telling operations can have keywords in the middle of them, not just at the front — which is how `DECLARE total DECIMAL` can name a thing and its type in one line without commas, and how a vocabulary can offer `SEND claim TO "finance"` if that reads better than `SEND claim, "finance"`. Part 3 is where those shapes get designed; here it is enough to know that a line with words scattered through it is still one operation, and that the words are part of its name.

How to tell them apart

Three questions, in order:

Is it part of a larger expression? Then it is asking. Only an answer can be assigned, compared, added to something, or returned.

Is it a whole line on its own? Then it is telling. It may or may not have brackets, and any keywords among its arguments belong to it.

Does its name appear in the vocabulary? If not, it is not an operation at all, and the program will not compile — which is the subject of chapter 8 and, in a different sense, of chapter 11.

Where the words come from

None of the operations in this chapter are part of BUBAS. `TOTAL_OF`, `APPROVE`, `REJECT` and `NOTE` exist because somebody building this application decided that this domain asks those questions and issues those instructions.

Which raises the obvious question: what happens when the rule you have been asked to write needs a question nobody thought to provide? That is a normal situation with a normal answer, and chapter 11 is about it.

The next chapter deals with the one kind of value this book has kept deferring: the domain objects themselves, which a program can hold but never open.

CHAPTER 5

Things the program cannot open

Some values a program holds are sealed: it can be given one, pass it on, and ask questions about it, but never reach inside. There is no `claim.employee.manager.email`*, and this chapter argues that the missing dot is a feature. What it means for you as a rule writer, and what to do when the question you want to ask has no operation yet.*

The fifth kind of value

Chapter 3 listed four kinds of value and then admitted there was a fifth. This is it.

`claim` is a `Report`. It is not a number, or text, or a yes-or-no; it is the expense claim itself, the same object the surrounding application was holding when it decided to run this rule. The program receives it, works with it, and hands it to operations that know what to do with it.

What the program cannot do is look inside.

That is not a restriction bolted on afterwards. There is no syntax in BUBAS for reaching into anything — no dot, no square brackets on a domain value, no field names, nothing. The capability is absent rather than forbidden, which is a distinction this book keeps returning to.

What you can do with one

Rather more than you might expect from a value you cannot open.

You can receive one as a parameter, which is how `claim` arrives. You can declare a variable of that type and assign to it. You can pass one to any operation that takes it, and you can pass the same one to several. An operation can hand you a new one, and you can keep it.

In other words a domain value behaves like any other value, in every respect except that its contents are somebody else's business. It is a thing you hold and give to people who know what to do with it, like a sealed envelope with a name on the front.

Rosencrantz and Guildenstern carry such an envelope. Hamlet sends them to England with a sealed letter; they never read it, they simply deliver it, and it is their own execution order. They are the perfect handlers of an opaque value — they hold it, they pass it on, and they cannot look inside.

The cautionary half of the analogy is worth keeping too. Everything that went wrong for them was decided by whoever wrote the letter, not by anything they did with it. A rule that passes a claim to `APPROVE` is exactly as safe, and exactly as dangerous, as whatever `APPROVE` was built to do — which is chapter 1's honest limit seen from the inside.

What you cannot do

Three refusals, and each one says something different.

You cannot reach inside.

```
line 3: unknown statement who
        who = claim.employee
```

That message is worth explaining, because it is the one you will meet if you try, and it does not mention dots. It cannot: there is no member-access syntax in the language for the compiler to complain about, so `claim.employee` is not a bad expression, it is not an expression. The line matches no statement the language knows, and that is what it reports. If you get this message, the question to ask is not "what is wrong with my dot" but "what operation gives me the employee".

You cannot turn one into text.

```
line 2: Report has no text form and cannot be added to a STRING; the
■ embedder can expose a domain-named function for it
        NOTE "deciding " + claim + " against " + limit
```

A number knows how to render itself and so does a boolean. A claim does not, because BUBAS has no idea what a claim is. If a message needs to identify the claim — and messages usually do — that is an operation somebody provides, named in domain terms, deciding for itself whether a claim reads as its number, its claimant, or both.

You cannot compare two of them.

```
line 3: Report and Report cannot be compared; an opaque value is a
■ black box, so the embedder decides what comparing two of them means
        IF claim = other THEN
```

Not even for equality. This surprises people more than the other two, and the message explains itself: comparing two claims is a domain question, not a language one. Are two claims equal when they have the same number? The same claimant and dates? Whoever built the vocabulary knows; BUBAS does not, and declines to guess. If your rule needs it, there is an operation for it or there needs to be.

Why it is total

The obvious middle path would be to expose *some* of a domain value. Let a rule read a claim's number and its date, say, but not go wandering through the object graph. Most systems that try this end up there.

BUBAS does not, for a reason that only becomes visible when you think about who reads the rules.

If a rule could see fields, then what a rule can know would be determined by the shape of a Java class — and Java classes grow. Someone adds a field for an unrelated feature, and the vocabulary has silently gained a word nobody decided to add, nobody documented, and nobody reviewed. The list of things a rule can find out would stop being a list anybody wrote.

With total opacity, that list is exactly the set of operations somebody chose to expose. It can be printed. It can be reviewed by the finance manager before a single rule is written. It changes only when a person changes it deliberately. Chapter 11 is about reading that list, and it only works as a complete account of the language because there is no second, accidental way in.

There is a second benefit, which Part 2 collects. A value nobody can open is a value nobody needs to *construct* in order to test a rule — a test can name one instead of building one. Total opacity is what makes that possible, and partial opacity would not.

The missing dot

It is worth sitting with what is absent, because the absence is the point.

`claim.employee.manager.email` cannot be written. Neither can the version of that expression which works fine for eleven months and then meets a claimant whose manager has left. Neither can the rule that quietly depends on a field somebody is about to rename, nor the one that reaches three objects deep into a structure whose shape was never meant to be a public interface.

Every one of those is a normal thing to write in a general-purpose language, and every one of them is a way for a business rule to acquire a dependency that its author did not think about and its reviewer cannot see. None of them are available here. What a rule depends on is the operations it calls, and those are on the page.

When you need something you cannot get

Sooner or later — usually sooner — you will want to ask a question the vocabulary does not have. The claim's submission date. Whether the claimant is a contractor. How many days the trip covered.

This is not a wall, and it is not a sign that you are using the language wrongly. It is the normal way a BUBAS vocabulary grows: somebody wants to write a rule, the rule needs a question, the question gets added. Chapter 11 covers how to find out what exists and how to ask for what does not, and Part 3 is where the person who builds it does their work.

What you should *not* do is work around it. If you find yourself computing something from three other operations because the direct question is missing, you have moved a piece of domain knowledge out of the vocabulary and into a rule, where the next person to write a similar rule will not find it. Ask for the operation.

What is coming

You now know all five kinds of value and both kinds of operation, which is the whole of what a BUBAS program is made of.

The next three chapters are about putting them together: deciding between branches, repeating work across the lines of a claim, and then — the one that changes how the rest of the book reads — what the compiler refuses to accept and why the list is shorter than you would expect.

CHAPTER 6

Deciding

`IF`, `ELSEIF`, `ELSE`, comparison, and `AND` / `OR` / `NOT` written as the words they are. How the order of tests is itself part of the policy — a claim caught by the meals cap before the total is ever considered is a decision somebody made, and it should be one they made on purpose.

One test

The simplest decision has one test and one thing to do about it:

```
IF total > limit THEN
    REJECT claim, "over the " + limit + " limit"
    RETURN FALSE
END IF
```

`IF`, a condition, `THEN`, some statements, `END IF`. The block is closed with words rather than with indentation or brackets, so a rule cannot change meaning because somebody's editor retabbed it.

The condition has to be a `BOOLEAN`. There are no truthy numbers here and no empty string standing in for false: a condition either answers yes or no, or the program does not compile.

More than two ways out

Real policy rarely divides in two. A claim can be fine, or need a manager, or be so far out that no manager should be signing it off. That is `ELSEIF`:

```
IF total > ceiling THEN
    REJECT claim, "over " + ceiling + " — needs a business case first"
    RETURN FALSE
ELSEIF total > limit THEN
    ESCALATE claim, "over the " + limit + " limit"
    RETURN FALSE
END IF
```

`ELSEIF` is one word. `ELSE IF`, `ELIF` and `ELSIF` are all rejected — the first of those because BUBAS matches statements a whole line at a time, so `ELSE IF`

`total > limit THEN` is two statements crowded onto one line and fails without needing a rule of its own.

The tests are tried in order and the first one that holds wins. Everything after it is skipped, including tests that would also have been true.

An `ELSE` for everything left

`ELSE` catches whatever reached the end without matching, and here is a rule using the full chain, with one extra input: whether the claim was marked urgent.

```
IF total > ceiling THEN
    REJECT claim, "over " + ceiling + " — needs a business case first"
    RETURN FALSE
ELSEIF total > limit AND NOT urgent THEN
    ESCALATE claim, "over the " + limit + " limit"
    RETURN FALSE
ELSEIF total > limit THEN
    NOTE "urgent, so cleared over the limit"
    APPROVE claim
    RETURN TRUE
ELSE
    APPROVE claim
    RETURN TRUE
END IF
```

Four ways out, and every one of them returns. Running it against three claims, with the middle one tried twice — once ordinary, once urgent:

```
RouteExpense(claim = report 1 (Alice), limit = 200.00, urgent = FALSE)
    approved report 1 (Alice) for 128.40
    => TRUE

RouteExpense(claim = report 2 (Erin), limit = 200.00, urgent = FALSE)
    escalated report 2 (Erin) — over the 200.00 limit
    => FALSE

RouteExpense(claim = report 2 (Erin), limit = 200.00, urgent = TRUE)
    urgent, so cleared over the limit
    approved report 2 (Erin) for 450.00
    => TRUE

RouteExpense(claim = report 3 (Frank), limit = 200.00, urgent = TRUE)
    rejected report 3 (Frank) — over 1000.00 — needs a business case
    ■ first
    => FALSE
```

Erin's claim is the same claim, at the same limit, both times. One `BOOLEAN` sends it to a manager or clears it.

Combining tests

Three words join and negate conditions, and they are words rather than symbols because a rule is meant to be read aloud:

- `AND` — both must hold
- `OR` — either will do
- `NOT` — the opposite

`total > limit AND NOT urgent` groups the way it reads. `NOT` binds tightest, then the comparisons, then `AND`, then `OR`. So `a > b AND c > d` compares first and combines second, which is what anybody would assume; and `a OR b AND c` means `a OR (b AND c)`, which is the one place people's assumptions differ, so bracket it if there is any doubt.

Brackets work as you expect and cost nothing. In a rule that somebody in finance has to approve, they are usually worth adding even where the precedence is on your side.

Comparing

Six comparisons: =, <>, <, >, <=, >=.

Note the first two. Equality is a single = — there is no == in this language — and "not equal" is <> rather than `!=`. Both are how people wrote them before programming borrowed the symbols from mathematics, and both read better to somebody who does not write code for a living.

Numbers compare against numbers, whole or decimal, mixed freely. Text compares too, character by character, so < on two strings asks which comes first alphabetically. Booleans can only be tested with = and <>, because asking whether `TRUE` exceeds `FALSE` is not a question. Domain values cannot be compared at all, for the reasons chapter 5 gave.

The order of the tests is the policy

This is the part worth slowing down for.

A chain of tests is not a set of independent rules. It is an ordered list, and moving one line changes what the rule does. In the stage-3 program you will meet in the next chapter, the meals cap is checked before the total, which means a claim of 75.00 against a limit of 200.00 can still fail — it never reaches the total test, because the meals test caught it first.

Is that right? It depends entirely on what the policy is meant to say, and that is not a question for whoever typed the rule. It is a question for whoever owns it, which is the whole reason this language exists.

So when reviewing a rule, read the tests in order and ask at each one: *if this fires, should the rest still be considered?* Most of the arguments worth having about a business rule turn out to be arguments about ordering, and here the ordering is on the page rather than distributed across four methods.

A test has to be a question

One thing an `IF` may not do is ask something the compiler can already answer.

`IF 1 = 1 THEN` is the obvious case and nobody writes it. The case people do write is a test on a value the rule worked out a few lines earlier — a flag set at the top, a cap assigned from a figure in the source. The compiler follows those values, so it knows the answer, so it refuses the test: one of the two branches cannot run, and a branch that cannot run is a mistake rather than a formality.

The remedy is nearly always the same, and it is worth knowing now rather than at the diagnostic. A value the rule sets for itself is decided; a value the rule is *given* is not. Anything the rule's behaviour should turn on belongs in the program's parameters, where the application supplies it and a test can set it either way. Chapter 8 works through the shape this takes in practice.

What is coming

One test at a time is enough for a claim considered as a single total. The next chapter opens the claim up and works through its lines one at a time, which needs a way of repeating — and brings back a second kind of value you cannot look inside.

CHAPTER 7

Repeating

Walking the lines of a claim: `FOR`, `DO WHILE`, `DO UNTIL`, *and leaving early with* `EXIT`. *Counting starts at one, because the first line on an expense claim is line 1 and nothing here is an array.*

The claim opens up

Until now a claim has been a single number: whatever `TOTAL_OF` said it came to. Real expense policy is not written that way. Receipts are required per line, not per claim. Meals have their own cap. A single hotel bill can be fine while the same money spent on dinners is not.

So the vocabulary gains a second kind of value — one line on a claim — and the operations to walk them:

Item

A single line on a claim: what was bought, from whom, for how much. Ask AMOUNT_OF, CATEGORY_OF, MERCHANT_OF and HAS_RECEIPT about one.

ITEM_COUNT(claim Report) -> INTEGER

How many lines a claim has.

ITEM_AT(claim Report, position INTEGER) -> Item

The line at a position on a claim. The first line is line 1.

AMOUNT_OF(line Item) -> DECIMAL

What a single line came to, in euro.

CATEGORY_OF(line Item) -> STRING

What kind of spending a line is: travel, meals, lodging.

MERCHANT_OF(line Item) -> STRING

Who the money was paid to on a line.

HAS_RECEIPT(line Item) -> BOOLEAN

Whether the claimant attached a receipt to a line.

That is the language describing itself. Every BUBAS vocabulary can produce a document like this, written for whoever will use it rather than for whoever built it — chapter 11 is about reading one. `Item` arrived the same way `Report` did: somebody decided this domain has lines, and made them askable.

An `Item` is opaque exactly as a `Report` is. You can hold one and ask it questions. You cannot open it, print it, or compare two of them.

Counting through the lines

```
FOR i = 1 TO ITEM_COUNT(claim)
    line = ITEM_AT(claim, i)

    IF AMOUNT_OF(line) > receiptFloor AND NOT HAS_RECEIPT(line) THEN
        REJECT claim, "no receipt for " + MERCHANT_OF(line)
        RETURN FALSE
    END IF

    IF CATEGORY_OF(line) = "meals" THEN
        meals = meals + AMOUNT_OF(line)
    END IF
END FOR
```

`FOR` takes a variable, a start, and an end, and runs the body once for each value from one to the other inclusive.

Counting starts at one. The first line on a claim is line 1, because that is what it is called by everyone who has ever looked at an expense claim. Zero-based counting is a convention that leaks out of how arrays are laid out in memory, and since this language never exposes an array, it has nothing to leak out of. `ITEM_AT(claim, 1)` is the first line.

Four smaller facts, each of which will eventually matter:

- The loop variable must be declared, as an `INTEGER`, before the loop.
- The start and end are worked out once, on entry. Changing what `ITEM_COUNT` would answer partway through does not lengthen the loop.
- The body may not assign the loop variable. Counting is the loop's job.

- `STEP` changes the stride, and may be negative: `FOR i = 10 TO 0 STEP -2`.

After the loop, the variable holds the first value that failed the test — which is occasionally useful and more often a trap, so prefer to carry anything you need out in a variable of your own.

Stopping early

A loop that has found what it was looking for should stop looking:

```
FOR i = 1 TO ITEM_COUNT(claim)
    line = ITEM_AT(claim, i)
    IF AMOUNT_OF(line) > limit THEN
        found = i
        EXIT FOR
    END IF
END FOR
```

`EXIT FOR` leaves the innermost enclosing `FOR`. There are no labels and no way to leave two loops at once; if you need that, carry a variable and test it.

Notice what the rule does with `found`. The loop records *which* line tripped it and gets out; the decision is made afterwards, in the open, where a reader can see it. Making the decision inside the loop would have worked too, and would have been harder to review.

```
FirstBigItem(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 128.40
    => TRUE

FirstBigItem(claim = report 2 (Erin), limit = 200.00)
    escalated report 2 (Erin) — line 1 is over the limit on its own
    => FALSE
```

When you are not counting

`FOR` is for when you know how many times. When the answer is "until something is true", the other form fits better:

```
DO WHILE i <= ITEM_COUNT(claim) AND running <= limit
    line = ITEM_AT(claim, i)
    running = running + AMOUNT_OF(line)
    i = i + 1
END DO
```

This one watches two things at once — how far through the claim it is, and whether the running total has already passed the limit — and stops on whichever happens first. A `FOR` could not express that without an `EXIT`.

```
RunningTotal(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 128.40
    => TRUE

RunningTotal(claim = report 4 (Carol), limit = 50.00)
    escalated report 4 (Carol) — passed the limit at line 2
    => FALSE
```

There are four shapes in total, and the condition may sit at either end:

- `DO WHILE condition` … `END DO` — tested before each pass, so the body may never run
- `DO UNTIL condition` … `END DO` — the same, negated
- `DO` … `END DO WHILE condition` — tested after each pass, so the body always runs once
- `DO` … `END DO UNTIL condition` — the same, negated

`EXIT DO` leaves a `DO` loop the way `EXIT FOR` leaves a `FOR`, and each only leaves a loop of its own kind.

The bottom-tested forms exist for a reason beyond taste, which chapter 8 explains: a loop whose body always runs at least once can satisfy the compiler that a variable has been set, and a top-tested loop cannot.

Everything together

The full stage-3 rule reads a claim line by line, checks receipts as it goes, accumulates the meals separately, and only then considers the total:

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS BOOLEAN
    DECLARE total DECIMAL
    DECLARE meals DECIMAL = 0.0
    DECLARE i INTEGER
    DECLARE line Item
    DECLARE ceiling DECIMAL FINAL = 1000.00
    DECLARE mealCap DECIMAL FINAL = 60.00
    DECLARE receiptFloor DECIMAL FINAL = 25.00

    FOR i = 1 TO ITEM_COUNT(claim)
        line = ITEM_AT(claim, i)

        IF AMOUNT_OF(line) > receiptFloor AND NOT HAS_RECEIPT(line)
        ■ THEN
            REJECT claim, "no receipt for " + MERCHANT_OF(line)
            RETURN FALSE
        END IF

        IF CATEGORY_OF(line) = "meals" THEN
            meals = meals + AMOUNT_OF(line)
        END IF
    END FOR

    IF meals > mealCap THEN
        ESCALATE claim, "meals came to " + meals + ", the cap is " +
        ■ mealCap
        RETURN FALSE
    END IF

    total = TOTAL_OF(claim)

    IF total > ceiling THEN
        REJECT claim, "over " + ceiling + " — needs a business case
        ■ first"
        RETURN FALSE
    ELSEIF total > limit THEN
        ESCALATE claim, "over the " + limit + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

```
ApproveExpense(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 128.40
    => TRUE

ApproveExpense(claim = report 5 (Dave), limit = 200.00)
    rejected report 5 (Dave) — no receipt for City Taxi
    => FALSE

ApproveExpense(claim = report 4 (Carol), limit = 200.00)
    escalated report 4 (Carol) — meals came to 75.00, the cap is 60.00
    => FALSE

ApproveExpense(claim = report 2 (Erin), limit = 200.00)
    escalated report 2 (Erin) — over the 200.00 limit
    => FALSE

ApproveExpense(claim = report 3 (Frank), limit = 200.00)
    rejected report 3 (Frank) — over 1000.00 — needs a business case
    ■ first
    => FALSE
```

Five claims, five different endings, and one of them worth arguing about. Carol's claim comes to 75.00 against a limit of 200.00 and is still not approved, because the meals cap is tested before the total and it never gets that far. Whether that is the right policy is exactly the kind of question the last chapter said belongs to the person who owns the rule — and here it is a single line's position on a page rather than an ordering buried in three methods.

The thing to watch

A loop is where a business rule most easily stops being a business rule.

If you find a loop building something up over many lines, sorting, comparing every line against every other, or keeping several running values at once and combining them at the end, stop and look at it. What you are writing may be an algorithm that wants to live behind a single operation, asked once. Chapter 9 gives that instinct a name and a test.

A loop has to be able to run, and to stop

Loops carry the same requirement as an `IF`, in two directions.

A loop that cannot run at all is refused: `FOR line = 5 TO 1` counts backwards without being told to, and its body is text nobody will ever execute.

So is a step of zero, which would never finish. So is a `DO WHILE` whose condition is decided before the first pass.

So is a loop with nothing in it. A body you have emptied out is a shape left behind after the contents were deleted, and the only reading under which it means anything is one where the work happens inside the condition — which is the kind of cleverness a rule should never contain. The same goes for an `IF` arm and an `ELSE`: write what belongs there, or delete the construct.

A loop that cannot stop is refused too — a condition that stays true with no `EXIT` anywhere inside it. `DO WHILE TRUE` is perfectly good BUBAS as long as something in the body leaves; what is refused is the version where nothing does.

Both come from the same idea as the rest of this chapter: the bounds of a loop are the part a reviewer checks, and a loop whose bounds settle the matter before the first pass is not a loop. The compiler says so rather than running it zero times and moving on.

What is coming

You have now seen every way a BUBAS program can be put together. The next chapter is the one that changes how the rest of the book reads: what the compiler refuses, and why the list of things that can go wrong is shorter than experience has taught you to expect.

CHAPTER 8

What will not compile

The refusals: a value read before it has been worked out, a constant assigned twice, a variable declared and never used. This chapter argues that these are not the compiler being fussy — with no null, one scope and no data structures, whole families of mistake have nowhere left to live. They are not caught. They are absent.

Not caught — absent

Every programming language has a list of mistakes it catches. What matters more is the list of mistakes it cannot make.

A rule that reads a total before working it out is not a bug BUBAS finds for you. It is a program that does not exist: it cannot be run, cannot be deployed, cannot reach a claimant. The distinction sounds like hair-splitting until you have spent an afternoon on a production incident caused by something that was, technically, caught — by a test somebody happened to write.

This chapter is the list. It is shorter than experience suggests it should be, and the last section explains why.

A value you have not worked out yet

```
line 4: 'total' is read before it is assigned (at 4:8)
        IF total > limit THEN
```

Declaring a variable does not give it a value. It reserves the name and the type, and until something assigns to it, reading it is an error.

Not zero. Not empty text. Not null. There is no value that means *not worked out yet*, so there is no way to accidentally compute with one, and no rule anywhere in your organisation that quietly treats a missing figure as nought.

The compiler traces every path through the program to prove this, which means it also catches the subtler version: a value assigned in one branch of an `IF` and read after the `END IF`, where the other branch never set it. That reads as

obviously fine when you write it and is obviously wrong when you draw the two paths out, which is what the compiler does for you.

Why there are four kinds of loop

Here is the same tracing making a distinction that looks pedantic and is not:

```
line 14: 'lastMerchant' is read before it is assigned (at 14:24)
        NOTE "last was " + lastMerchant + ", limit " + limit
```

The loop sets `lastMerchant` on every pass. But a `DO WHILE` tests its condition *before* the first pass, so a claim with no lines runs the body zero times, and `lastMerchant` is read having never been set. The compiler will not take the chance, and it is right not to: an empty claim is exactly the input nobody tests by hand.

Move the test to the bottom and the same program compiles:

```
DO
    line = ITEM_AT(claim, i)
    lastMerchant = MERCHANT_OF(line)
    i = i + 1
END DO WHILE i <= ITEM_COUNT(claim)
```

The body always runs at least once, so the assignment is guaranteed, and the compiler can see it.

That is what the four loop shapes are for. They are not stylistic variants; the position of the condition changes what the compiler can prove.

A constant is constant

```
line 4: 'mealCap' is final and cannot be changed
        mealCap = limit
```

`FINAL` on a declaration means the value is fixed at that line and never changes again. Parameters are final too — nothing can reassign what the program was given.

This matters more in a business rule than in ordinary code. The policy numbers in a rule — a cap, a floor, a threshold — are the part a reviewer reads

most carefully, and `FINAL` is a promise that what they read at the top is what applied at the bottom. A rule that adjusts its own cap partway down is one nobody can review by reading it once.

A decision that is already made

A rule that tests something it has just decided for itself is not testing anything. Here is the shape it usually arrives in — a switch, added so the rule could be tightened without touching the application:

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS BOOLEAN
    DECLARE strict BOOLEAN
    DECLARE cap DECIMAL

    strict = TRUE
    cap = limit

    IF strict THEN
        cap = limit - 100.00
    END IF

    IF TOTAL_OF(claim) > cap THEN
        REJECT claim, "over the " + cap + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

```
line 8: this condition is always TRUE, so nothing after this arm can
■ run; delete the IF and keep its body
        IF strict THEN
```

Nothing in that program is written as a constant. `strict` is an ordinary variable, assigned on an ordinary line, and read four lines later. The compiler followed the value: at the `IF`, `strict` is `TRUE`, so the test has an answer, so the `IF` decides nothing and the `END IF` is decoration.

This is worth refusing rather than allowing, because of what the line looks like six months later. A reviewer reading `IF strict THEN` sees a rule with two modes. There is one, and finding out which means scrolling up. Worse is the version where the assignment moves — a rule that quietly stopped having a strict mode at all still reads as though it has one, and the tests that covered the other

branch still pass, because the branch they cover is the only one there is.

The compiler is not objecting to the value. It is objecting to a question with no question in it.

The switch belongs where the application can turn it:

```
PROGRAM ApproveExpense(claim Report, limit DECIMAL, strict BOOLEAN)
■ RETURNS BOOLEAN
    DECLARE cap DECIMAL

    cap = limit

    IF strict THEN
        cap = limit - 100.00
    END IF

    IF TOTAL_OF(claim) > cap THEN
        REJECT claim, "over the " + cap + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

```
ApproveExpense(claim = report 1 (Alice), limit = 200.00, strict =
■ FALSE)
    approved report 1 (Alice) for 128.40
    => TRUE

ApproveExpense(claim = report 1 (Alice), limit = 200.00, strict =
■ TRUE)
    rejected report 1 (Alice) — over the 100.00 limit
    => FALSE
```

One program, one claim, two answers. `strict` is now something the program is *told*, which is the only thing in a rule the compiler genuinely cannot know — and so the `IF` is a question again, both branches are reachable, and a test can cover each.

That is the general answer whenever this refusal appears. A value the rule works out for itself is known where it is read; a value handed to the rule is not. If a test has to vary something, it is a parameter.

The same reasoning reaches arithmetic that cannot come out. `1 / 0` is refused where it is written, whether or not anything could reach the line — a line that cannot succeed is not made acceptable by being hard to get to — and so is a total that would overflow, and a cap worked out from figures that leave nothing to work out.

Every path must answer

```
line 1: this program declares RETURNS BOOLEAN but can reach its end
■ without returning a value
    PROGRAM ApproveExpense(claim Report, limit DECIMAL) RETURNS
    ■ BOOLEAN
```

A program that says it answers `BOOLEAN` must answer on every path out of it. Falling off the end is not an implicit no.

The common shape of this mistake is a chain of tests where every branch returns except the one nobody thought about, which is usually the ordinary case. Here the program says so at compile time instead of returning something unhelpful in production.

The loop counts, and you do not

```
line 9: 'i' is the variable of an enclosing FOR loop and cannot be
■ assigned inside it
            i = i + 1
```

Inside a `FOR`, the counter belongs to the loop. Skipping ahead by assigning to it — a classic way to make a loop that reads as though it visits every line and does not — is refused outright. If you want to skip lines, test for the ones you want and leave the counting alone.

Types, including the ones BUBAS cannot see

```
line 4: AMOUNT_OF takes Item for 'line', but was given Report
        spent = AMOUNT_OF(claim)
```

Numbers, text and booleans are checked as you would expect. So are the domain values, and that is the interesting part: BUBAS has no idea what a

`Report` or an `Item` *is*, and still knows they are not interchangeable. It knows because somebody registered two different names, and two names are two types.

Note what the message gives you: the type wanted, the type supplied, and the name of the thing it was wanted for. That last part comes from the vocabulary rather than from your program, which is one of several places in this book where the care taken in Part 3 shows up as a better experience in Part 1.

An answer nobody keeps

```
line 4: TOTAL_OF returns a value, so it cannot stand alone as a
■ statement; a result would be discarded silently
        TOTAL_OF claim
```

Asking a question and dropping the answer is refused, so a line that looks like it does something always does. Chapter 4 covered this; it belongs on the list.

A variable nobody reads

Declaring a variable and never reading it is also an error — `'headroom' is declared but never read`.

This is the refusal people push back on hardest, and it earns its place. In a business rule, an unread variable is nearly always the fossil of a policy change: a figure that used to be compared against something, left behind when the comparison moved or was deleted. It is a reviewer's false friend, because it reads as though it still participates. Removing it is a two-second edit; leaving it is a rule that lies to the next person.

What is not on the list

The refusals above are the interesting half. The other half is a set of mistakes that never come up at all, because nothing in the language can express them.

There is no null, so there is no dereferencing one. There is no array indexing on a domain value, so there is no reading past the end of one. There are no nested scopes, so no variable quietly shadows another and no rule reads the wrong one of two things with the same name. There are no data structures, so nothing can be aliased, mutated by somebody else's code, or left in a half-built state. There is

no inheritance, so nothing calls an override you did not know existed. There is no concurrency in a program, so nothing races.

None of these are caught. There is nothing to catch.

Why the list is short

It would be easy to read this chapter as a language being strict. That is not quite it.

Most of what a compiler for a large language checks is defending against the size of the language. Null checks exist because there is null. Escape analysis exists because there are references. Immutability rules exist because there is shared mutable state. Take those away and the checks are not passed — they become unnecessary.

What is left is a short list of things that are genuinely about the rule: did you work out this figure before using it, does every path answer, is the constant constant. Those are questions the person who owns the policy would ask too, if they knew to ask them. Everything else the compiler would have had to check has been designed out.

What is coming

One thing on this list has been deferred twice now: the single aggregate BUBAS does have. The next chapter is about arrays — what they can do, and why wanting one is usually a signal worth listening to.

CHAPTER 9

Arrays, and wanting one

The one aggregate BUBAS has, what it can and cannot do, and a warning. Wanting a data structure is usually a signal that you have stopped writing a business rule and started writing an algorithm — which is a reasonable thing to want, and belongs behind an operation rather than in the rule.

The one aggregate

BUBAS has arrays. They are the only way to hold more than one value in a single name, and they are deliberately plain.

An array is declared with its size between the name and the type:

```
DECLARE totals[2] DECIMAL
```

The size can be any `INTEGER` expression and is worked out once, when the declaration runs, so `DECLARE lines[ITEM_COUNT(claim)] Item` is legal and sizes itself to the claim.

What you can rely on:

- Every element starts at its type's default — zero, empty text, `FALSE` — so an array is never half-built.
- `LENGTH(totals)` gives the size.
- Arrays are one-dimensional. There are no arrays of arrays.
- An array cannot be `FINAL`, and neither can an element.
- **Indices start at zero.**

That last point deserves its own paragraph, because the previous chapters said the opposite.

Two ways of counting, and why

`ITEM_AT(claim, 1)` is the first line of a claim. `totals[0]` is the first element of an array. In the same program.

That looks like an inconsistency and is actually the distinction the language is built on. `ITEM_AT` is a domain operation — somebody decided expense claims have lines and that people number them from one, because they do. An array is a language mechanism, and it counts from zero the way machine structures have counted from zero for fifty years.

The mismatch is uncomfortable, and it should be. It is the language telling you that you have stepped out of the domain and into machinery. Which brings us to the point of the chapter.

A rule that wants an array

Here is a real requirement: meals have one cap, travel has another, and a claim breaching either goes to a manager. Stage 3 has everything needed to write it — lines, categories, amounts — so let us write it.

```
FOR i = 1 TO ITEM_COUNT(claim)
    line = ITEM_AT(claim, i)

    slot = -1
    IF CATEGORY_OF(line) = "meals" THEN
        slot = 0
    ELSEIF CATEGORY_OF(line) = "travel" THEN
        slot = 1
    END IF

    IF slot >= 0 THEN
        totals[slot] = totals[slot] + AMOUNT_OF(line)
    END IF
END FOR
```

It works. It is also doing several things that should make you uneasy.

The rule now contains a **mapping from category names to array positions**, invented on the spot. Nothing in the domain says meals are slot 0 and travel is slot 1; a rule made that up, and the next rule will make up a different one. The two caps arrive as separate parameters and have to be compared against the right positions — `totals[0]` against `mealCap` — a correspondence held together by nothing but the author getting it right twice.

Add a third category and you edit the size, the mapping, a parameter, and a comparison, in four places, with nothing to tell you if you miss one.

And the reviewer — the person this whole language exists for — now has to hold a slot-numbering scheme in their head to check a policy about dinners.

The same rule, asked instead

The vocabulary gains one operation:

TOTAL_FOR(claim Report, category STRING) -> DECIMAL

What one category of spending on a claim comes to, in euro.

And the rule becomes the whole of this:

```
PROGRAM CategoryCaps(claim Report, mealCap DECIMAL, travelCap
■ DECIMAL) RETURNS BOOLEAN
    IF TOTAL_FOR(claim, "meals") > mealCap THEN
        ESCALATE claim, "meals came to " + TOTAL_FOR(claim, "meals")
        RETURN FALSE
    END IF

    IF TOTAL_FOR(claim, "travel") > travelCap THEN
        ESCALATE claim, "travel came to " + TOTAL_FOR(claim, "travel")
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

No array, no loop, no mapping, no positional correspondence. A reader who knows nothing about programming can check it against the policy document line by line.

Both versions are compiled and run against the same claims by the same test, which asserts they reach identical decisions and produce identical messages. This is what they both say:

```
CategoryCaps(claim = report 1 (Alice), mealCap = 60.00, travelCap =
■ 500.00)
    approved report 1 (Alice) for 128.40
    => TRUE

CategoryCaps(claim = report 4 (Carol), mealCap = 60.00, travelCap =
■ 500.00)
    escalated report 4 (Carol) — meals came to 75.00
    => FALSE

CategoryCaps(claim = report 2 (Erin), mealCap = 60.00, travelCap =
■ 500.00)
    approved report 2 (Erin) for 450.00
    => TRUE
```

Nothing was given up. The work did not vanish — it moved behind `TOTAL_FOR`, where it is written once, tested once, and named in the language of the domain.

The test

You will not always have someone available to add an operation the moment you want one, so it helps to recognise the signal early. **Wanting an array is usually the signal.**

More precisely, ask what the array is for. If you are accumulating something across the lines of a claim in order to consult it afterwards, you are building an intermediate result, and an intermediate result is a thing the domain has no name for. That is the definition of having left business logic.

A few shapes that mean the same thing:

- Building up values to compare against each other at the end
- Sorting anything
- Comparing every line against every other line
- Keeping several running figures and combining them afterwards
- Any mapping from a domain name to a number you invented

None of these are forbidden and none of them are wrong. They are all *algorithms*, and algorithms belong behind an operation, written in Java by somebody who can test them properly — which is chapter 23's subject.

When an array is the right answer

Not never, or the language would not have them.

The honest case is a domain that is genuinely plural about something, with operations that hand you the plural thing. How many people must sign a claim is a fact about the approval policy, not something a rule should work out — so the vocabulary answers the count and fills the list. The first is a question and is a function; the second writes into a variable and is a statement, for the reason chapter 4 gave: a write belongs to the shape of the line, not to an argument whose result happens to be thrown away.

APPROVER_COUNT(claim Report) -> INTEGER

How many people have to approve a claim.

APPROVERS OF _ INTO _

```
APPROVERS OF {expression/Report:claim} INTO {var/ARRAY/STRING:into}
```

Fills an array with everyone who has to approve a claim, in signing order.

`APPROVERS OF claim INTO signers` reads as an instruction because it is one — chapter 24 is where statements like this are built. A rule that has to notify each of them then holds an array, and loops:

```
PROGRAM NotifyApprovers(claim Report, limit DECIMAL) RETURNS BOOLEAN
    DECLARE signers[APPROVER_COUNT(claim)] STRING
    DECLARE i INTEGER
    DECLARE total DECIMAL

    total = TOTAL_OF(claim)

    IF total <= limit THEN
        APPROVE claim
        RETURN TRUE
    END IF

    APPROVERS OF claim INTO signers

    FOR i = 0 TO LENGTH(signers) - 1
        NOTIFY signers[i]
    END FOR

    ESCALATE claim, "needs " + LENGTH(signers) + " signatures"
    RETURN FALSE
END.
```

```
NotifyApprovers(claim = report 1 (Alice), limit = 200.00)
    approved report 1 (Alice) for 42.50
    => TRUE

NotifyApprovers(claim = report 2 (Bob), limit = 200.00)
    told the line manager
    told the finance director
    told the board
    escalated report 2 (Bob) — needs 3 signatures
    => FALSE
```

Nothing here is a smell. There is no mapping the rule invented, no positional correspondence held together by care, and nothing a reviewer must hold in their head. The array's length came from the policy, its contents came from the policy, and the rule's only opinion is that each of them should be told.

Notice also how little the rule knows. It never learns why three signatures are needed rather than one; that decision stayed where it belongs. Had the rule counted approvers itself — one over a thousand, three over ten thousand — the array would have been the same shape and the chapter's warning would apply in full.

The distinction is whether the collection came *from* the domain or was assembled *by* your rule. The first is fine. The second is the signal.

What is coming

The vocabulary has grown from three operations to a dozen without ever changing one, which is what the last several chapters have quietly been demonstrating.

The next chapter adds an operation unlike any so far: one that does not calculate an answer but asks a model for an opinion, and will not necessarily give the same one twice.

CHAPTER 10

Operations that consult a model

An operation that returns how unusual a line of spending looks, on a scale of one to ten, and is backed by a model rather than a calculation. The score is advice; the threshold you compare it against is in your program, where you own it. What it costs to have an operation that will not give the same answer twice — which is where Part 2 begins.

An operation with an opinion

Every operation so far has calculated something. `TOTAL_OF` adds up lines. `HAS_RECEIPT` looks to see whether a file is attached. Ask either one twice and you get the same answer, because there is a right answer and it found it.

This one is different:

ANOMALY_SCORE_OF(line Item) -> INTEGER

How unusual a line of spending looks, from 1 (ordinary) to 10 (very unusual). It is an opinion, not a decision: the rule decides what score is too high.

There is no right answer to how unusual a dinner looks. Behind that operation is a model — the kind of thing that reads a line of spending and produces a judgement, the way an experienced person in accounts payable would. It might notice that the amount is large for its category, that no receipt is attached, that the merchant is one nobody has claimed against before, that the date is a Saturday. It does not explain itself, and it will not necessarily say the same thing tomorrow.

Nothing about how a BUBAS program uses it is different. It is a function; you call it; it answers an `INTEGER`. Everything strange about it is behind the boundary.

What the rule does with an opinion

```
PROGRAM ScreenExpense(claim Report, limit DECIMAL, flagAt INTEGER)
■ RETURNS BOOLEAN
    DECLARE i INTEGER
    DECLARE line Item
    DECLARE worst INTEGER

    worst = 0

    FOR i = 1 TO ITEM_COUNT(claim)
        line = ITEM_AT(claim, i)

        IF ANOMALY_SCORE_OF(line) > worst THEN
            worst = ANOMALY_SCORE_OF(line)
        END IF
    END FOR

    IF worst >= flagAt THEN
        ESCALATE claim, "a line scored " + worst + ", and this rule
        ■ flags at " + flagAt
        RETURN FALSE
    END IF

    IF TOTAL_OF(claim) > limit THEN
        ESCALATE claim, "over the " + limit + " limit"
        RETURN FALSE
    END IF

    APPROVE claim
    RETURN TRUE
END.
```

The rule walks the lines, keeps the worst score it saw, and compares it against `flagAt`.

That comparison is the whole point of this chapter, so it is worth being blunt about it. **The model does not decide anything.** It offers a number. The rule decides what number is too high, and the rule is on the page, in the language of the business, where the person accountable for expense policy can read it, argue with it, and change it.

Here is the same claim, twice, with only that threshold moved:

```
ScreenExpense(claim = report 1 (Alice), limit = 2000.00, flagAt = 8)
    approved report 1 (Alice) for 128.40
    => TRUE

ScreenExpense(claim = report 9 (Quentin), limit = 2000.00, flagAt = 8)
    escalated report 9 (Quentin) — a line scored 8, and this rule
    ■ flags at 8
    => FALSE

ScreenExpense(claim = report 9 (Quentin), limit = 2000.00, flagAt = 9)
    approved report 9 (Quentin) for 411.00
    => TRUE
```

Quentin's dinner scores 8 both times. At `flagAt = 8` the claim goes for review; at `flagAt = 9` it is paid. Nobody retrained anything. A person changed a number in a rule, and can be asked why.

Why a score and not a verdict

It would have been easy to expose this differently. The vocabulary could have offered `SHOULD_APPROVE(claim) -> BOOLEAN`, and the rule would have been one line long.

That version is worse, and the reason is not about accuracy.

With a score, the policy — what counts as too unusual — lives in the rule. It is visible, it is version-controlled, it is reviewable by someone in finance, and when the auditor asks why this claim was flagged and that one was not, the answer is a line somebody wrote on purpose.

With a verdict, the policy lives behind the operation. It is not invisible — it is in the Java, and an engineer can read it, review it and explain it. But the person who owns expense policy cannot, and they are the one the auditor will ask. The policy has moved out of the artefact they can hold you to and into one they need an interpreter for.

And your BUBAS program, which existed so that a subject-matter expert could own the rule, has been reduced to glue between a service and a database.

Generalised, and worth remembering when you are asked to add an operation:

> ***Advisory outputs — scores, classifications, extractions — leave the decision with the expert. Verdict outputs move it to the model.***

That is not an argument against models. It is an argument about where the seam goes.

What it costs

An operation that will not give the same answer twice has consequences, and they should be stated plainly rather than discovered.

A rule using it is no longer reproducible. Run it on the same claim next week and it may decide differently, because the model changed even though the rule did not. Every other operation in this book is stable in a way this one is not.

Calls cost something. A model behind an operation may take a second and may cost money per call. Look again at the rule above: it calls `ANOMALY_SCORE_OF` twice for every line, once to compare and once to store. On a claim with twenty lines that is forty calls where twenty would do. With `TOTAL_OF` nobody would care. Here it is worth assigning the score to a variable first — one of the few places in this book where how a rule is written affects anything but its readability.

It can fail in a way arithmetic does not. A service can be slow, or down, or answer nonsense. What should the rule do then? That question belongs to whoever exposes the operation, and chapter 30 is about the two audiences for the answer.

And it cannot be tested by running it. This is the big one, and it is the bridge to the next part of the book.

The thing Part 2 is about

Every test you might write for the rule above runs into the same wall. You cannot assert that a claim is escalated, because you cannot know what the model will say about it. You cannot even assert that it is *not* escalated, for the same reason.

The usual advice — mock the things your code depends on — is advice that most people quietly ignore for databases, because you can just run a database. Here you cannot just run the model. Mocking stops being tidiness and becomes the only way this rule can be checked at all.

And what you are checking, once the score is pinned to a known value, is not the model. It is your rule: that a score of 8 is escalated when the threshold is 8, that a score of 7 is not, that the worst line wins rather than the first. Those are policy questions with right answers, and they are exactly what the person who owns the policy would want checked.

Part 2 is how.

The honest limit

One thing this chapter should not leave you believing.

The vocabulary bounds what a program can *name*. It does not bound what a named thing *does*. An operation described as returning an opinion about a line of spending could, behind that boundary, send the whole claim to a third party, log the claimant's name somewhere it should not be, or cost a euro a call. The description is what somebody wrote; the behaviour is what somebody built.

This is true of every operation in the book, and it has been said before. It is worth repeating here because a model-backed operation is the one where the gap between the description and the behaviour is the largest, the hardest to inspect, and the most likely to change without anybody editing a line of code. The review checksum that chapter 11 mentions will not notice. Nothing in BUBAS will.

What is coming

You have now met every kind of operation a BUBAS vocabulary can offer. The last chapter of this part is about the vocabulary itself: how to find out what a language you have been handed can actually do, and what to do when it cannot do what you need.

CHAPTER 11

Knowing what you can say

Every BUBAS language can describe itself: a document listing every operation, what it answers, and what it is for, written for you rather than for a programmer. How to read it, how to tell whether a rule you have been asked to write is expressible, and how to ask for an operation that does not exist yet. That last one is a normal request, and Part 3 is where it gets answered.

The question

You have been handed a language and asked to write a rule in it. Before you can start, you need to know what it can say.

In a general-purpose language that question has no good answer — the honest reply is "almost anything, go and read the codebase." In BUBAS it has an exact one, because the set of words a program can use is finite and somebody wrote it down. Every sealed language can produce a document listing all of it.

That document is not generated from comments, or from a wiki somebody keeps up to date. It comes from the language itself, so it cannot describe an operation that does not exist and cannot omit one that does.

Values

It opens with the things your rules will hold but never open:

Values

A script may hold one of these, pass it and store it in an array. It can never look inside: every question about one is a function.

Report

One employee's expense claim for a trip or a period. A program is given one to decide about; ask TOTAL_OF what it comes to.

Item

A single line on a claim: what was bought, from whom, for how much. Ask AMOUNT_OF, CATEGORY_OF, MERCHANT_OF and HAS_RECEIPT about one.

Notice what the descriptions do. They do not say what a `Report` is made of, because you cannot reach into one. They tell you what it *is* in the business, and which operations to ask about it. That is the right shape for a reader who will never see the Java, and it is the shape to insist on when you review a vocabulary somebody has drafted for you.

Questions you can ask

Then the functions — everything the language can find out:

TOTAL_FOR(claim Report, category STRING) -> DECIMAL

What one category of spending on a claim comes to, in euro.

The heading is the whole signature: the name, what it needs and of what type, and what it answers after the arrow. `TOTAL_FOR(claim Report, category STRING) -> DECIMAL` tells you it wants a claim and some text and gives back a decimal, which is everything you need to use it correctly.

An entry with no arrow answers nothing and is written as a statement rather than used in an expression — which is chapter 4's distinction, visible in the document.

Things you can tell it to do

Then the statements:

REJECT _, _

```
REJECT {expression/Report:claim}, {expression/STRING:reason}
```

Refuses a claim, recording the reason the claimant will be shown.

The heading shows the shape with the blanks marked, and the block underneath spells it out precisely. That block is written for the person who built the language, and you can mostly skim it, but two parts of it are worth learning to read.

`{expression/Report:claim}` means *a claim goes here, and it must be a* `Report`. Anything of that type will do — a variable, or something a function just answered.

Some entries end with a line like `Leaves a value in: name`. That is the document telling you that the statement *writes* to the variable in that position rather than only reading it. `DECLARE` does; `REJECT` does not. It matters when you are working out whether a variable has been given a value yet, which is the rule chapter 8 spent its first section on.

You will also notice that `DECLARE` and `=` are in this list, alongside `APPROVE` and `REJECT`. That is chapter 4's point made concrete: they are operations somebody installed, not syntax the language was born with.

What the document does not tell you

Two limits, and both matter.

It describes what an operation is for, not what it does. The description is prose somebody wrote. Behind `ANOMALY_SCORE_OF` there is code, and the code could do more than the sentence admits. For most vocabularies, written by your own colleagues for your own application, that gap is uninteresting. It is worth remembering it exists.

It cannot tell you whether the descriptions are current. An operation's behaviour can change without its description changing. There is a way to record that a vocabulary was reviewed at a particular shape, so that adding or removing an operation shows up as a review that needs redoing — chapter 26 covers it — but nothing detects a description that has quietly stopped being true. That is a job for the people, not the tooling.

Can I write the rule I have been asked to write?

This is the practical use of the document, and it is worth doing before writing anything.

Take the policy you have been handed and go through it clause by clause, asking of each: *which operation answers this?* Most clauses will map onto something. The ones that do not are what you have learned.

Three ways it can go.

The question exists under another name. You want "how much was spent on food" and the vocabulary offers `TOTAL_FOR(claim, "meals")`. Fine.

The question can be built from others. You want the average per line, and there is `TOTAL_OF` and `ITEM_COUNT`. Also fine, if the arithmetic is genuinely part of the rule — but see the warning below.

The question is not there at all. You want to know whether the claimant is a contractor, and nothing in the vocabulary knows anything about people. Nothing you can write will get you there, because there is no way in. Stop, and ask.

Ask, rather than working around

When a question is missing, there is nearly always a way to fake it. You can compare a merchant name against a list of strings you type into the rule. You can infer a trip's length from the number of lodging lines. You can decide that anything over some amount is probably a contractor.

Do not.

Every one of those moves a piece of domain knowledge out of the vocabulary — where it is written once, named, reviewed, and shared by every rule — into one rule, where the next person to need it will not find it and will invent their own slightly different version. Two rules that disagree about what a contractor is, in a way nobody notices for a year, is a worse outcome than waiting a week for an operation.

The instinct to work around it is strong, because working around it feels like getting on with the job. It is worth resisting precisely as strongly.

What makes a good request

The person who will build the operation is doing Part 3's work, and the request that helps them most is not "please add `IS_CONTRACTOR`."

Tell them the **policy clause** you are trying to express, in the words it was given to you. Tell them **what you would do with the answer** — branch on it, put it in a message, compare it against a threshold. Tell them **what kind of**

answer you expect: a yes or no, a number, one of a few known words.

That is enough for them to decide the shape, and the shape is the part that is hard to change later. A question exposed as a boolean when it should have been a score is chapter 10's mistake, and it is much easier to avoid before the operation exists than after twenty rules depend on it.

End of Part 1

You can now read a BUBAS program, write one, predict what the compiler will refuse, find out what a language you have been given can do, and ask for what it cannot.

What you cannot yet do is show that a rule is *right*. Reading a rule and agreeing with it is worth a great deal, and it is not the same as demonstrating that it does what you think on the claims it will actually meet — including the awkward ones nobody thinks of until they arrive.

Part 2 is about that, and it is written in the same language, for the same reader.

PART 2

Testing

CHAPTER 12

Why the test must be readable too

A rule reviewed by the person accountable for it needs a test that the same person can check. A test written in a language they cannot read moves the review straight back to where chapter 1 found it. BUNIT tests are written in BUBAS for exactly this reason.

The review, one step further on

Chapter 1 described a review that did not happen: a rule nobody accountable could read, approved by somebody who could only check that it looked reasonable. Part 1 fixed that. The rule is now eleven lines that a finance manager can read and argue with.

Now ask the same question about the test.

A rule can be read and agreed with and still be wrong. Agreement means the reader followed what the rule says; it does not mean they worked out what it does on a claim of exactly 200.00 against a limit of exactly 200.00, or on a claim with no lines, or on the one that arrives at the end of the budget year. Those are the cases that matter, and nobody finds them by reading.

So there is a second artefact — the test — and it carries the answers to precisely the questions the reviewer could not answer by reading. If that artefact is written in Java, the review has moved right back to where chapter 1 found it. The person who knows whether the threshold is inclusive cannot check that anybody encoded it as inclusive.

What a test looks like here

```
PROGRAM UnderTheLimitIsApproved
    "TOTAL_OF" WITH ARGS("Alice's claim") RETURNS 128.40
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "Alice's claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS TRUE
    "APPROVE _" WAS CALLED WITH ARGS("Alice's claim")
    "REJECT _, _" WAS NOT CALLED
END.
```

That is a complete, running test of the rule from chapter 2, and it is written in BUBAS. Not in a BUBAS-flavoured configuration file, and not in something that generates BUBAS — in the language, with the same statements, the same quoting, the same `END.`

You can read it for the same reason you could read the rule. The claim comes to 128.40, the limit is 200.00, the answer should be yes, the claim should be approved, and nothing should be refused.

More to the point, you can *disagree* with it. Should a claim of exactly 200.00 pass? This test does not say. Somebody should decide, and that somebody is not whoever typed the test.

Reading a test is not the same as reading a rule

There is a real difference worth naming, because it is where the value is.

A rule tells you what the policy is. A test tells you what the policy **does on one specific case**, chosen by somebody who was thinking about where it might go wrong. A suite of them is a list of cases somebody thought about — which is exactly the artefact a reviewer wants and never normally gets.

When the tests are readable, "have we covered the case where a claim is exactly at the limit?" stops being a question for engineers and becomes a question anybody can answer by looking.

The three things a test says

Every BUNIT test has the same shape, and it maps onto how anybody would describe a case out loud.

What the world was like. The claim came to 128.40. Nothing else about the world matters, so nothing else is mentioned.

What happened. The rule ran, with this claim and this limit.

What should be true afterwards. It said yes, it approved the claim, and it did not refuse anything.

Chapter 13 goes through it line by line. What matters here is that all three are in the language of expense approval rather than the language of testing. There are no fixtures, no setup methods, no builders, no assertion library. There is a claim, a limit, and what should have happened to them.

Running them

```
3/3 passed
PASSED UnderTheLimitIsApproved
PASSED OverTheLimitIsRefused
PASSED ExactlyAtTheLimitIsApproved
```

Three cases, three passes, and the names are sentences rather than method names. A report like that can go in front of the person who owns the policy without translation, which is the whole argument of this part in one screen.

What this part covers

The next chapter takes a single test apart. After that:

Standing in for the world. Your rule calls operations that load claims, charge budgets and send messages, and a test cannot have those really happen. What replaces them, and what that costs.

Tokens. A claim is a value you cannot construct. Tests name them instead.

Matching arguments. Saying what matters about a call and leaving the rest alone.

Leaving some of it real. When standing in for less makes a test more honest, and when it hides what you meant to check.

Mocks that cannot be right. The framework refuses some tests before running them, and the reasons are worth understanding rather than working around.

Testing what cannot be tested. The model-backed operation from chapter 10, which cannot be tested by running it at all.

A suite that stays honest. What to cover, and a coverage claim that a finite vocabulary makes possible.

Still no Java. The person who owns the rule can read every page of this part, and the engineer who will embed the language needs all of it too.

CHAPTER 13

A first test

A complete test of the rule from chapter 2: the claim it is given, what it should decide, and what the runner prints when it agrees and when it does not. Enough to write tests for rules you already have.

The whole test

```
PROGRAM UnderTheLimitIsApproved
    "TOTAL_OF" WITH ARGS("Alice's claim") RETURNS 128.40
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "Alice's claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS TRUE
    "APPROVE _" WAS CALLED WITH ARGS("Alice's claim")
    "REJECT _, _" WAS NOT CALLED
END.
```

Fourteen lines, testing the eleven-line rule from chapter 2. Taken apart below, in the order it runs.

The name is a sentence

```
PROGRAM UnderTheLimitIsApproved
```

A test is a BUBAS program like any other, and its name is what the report prints. Name it after the case, not after the rule: `UnderTheLimitIsApproved` tells a reader what failed when it fails, where `TestApproveExpense1` tells them nothing.

Standing in for the world

```
"TOTAL_OF" WITH ARGS("Alice's claim") RETURNS 128.40
"APPROVE _" IS MOCKED
"REJECT _, _" IS MOCKED
```

The rule calls three operations. In a test none of them may really happen — `TOTAL_OF` would want a real claim, and `APPROVE` would pay somebody — so each is replaced.

`TOTAL_OF` is replaced by an **answer**: asked about Alice's claim, say 128.40. That is the case this test is about.

`APPROVE` and `REJECT` answer nothing, so there is nothing to say except that they are stood in for. `IS MOCKED` means *let this be called, do nothing, and remember that it was.*

The quoted names are the shapes from the vocabulary. A function is its name; a statement is its name with _ for each thing it takes, which is exactly how the vocabulary document of chapter 11 lists it. `REJECT` takes two, so it is `"REJECT _, _"`.

What the rule is given

```
ARGUMENT "claim" IS "Alice's claim"
ARGUMENT "limit" IS 200.00
```

One line per parameter, in the rule's own words.

`limit` is a decimal and reads as one. `claim` is a `Report` — one of those values chapter 5 said a program can hold but never open — and a test cannot construct one either. So it names one instead. `"Alice's claim"` is a label for a claim that does not exist, which is enough, because the rule never looks inside it. The next chapter is about why that works.

Note that the same label appears in the mock above. That is not decoration: it is how the test says *when you are asked about this particular claim, answer 128.40.*

Running it

```
RUN
```

Everything above describes the world. `RUN` is the moment the rule executes. Everything below is about what happened.

The order is not a style rule. An expectation written before `RUN` is refused, because it would be asking about something that has not happened — chapter 18 is about that check and the others like it.

What should be true afterwards

```
RESULT IS TRUE
"APPROVE _" WAS CALLED WITH ARGS("Alice's claim")
"REJECT _, _" WAS NOT CALLED
```

Three claims about the run.

`RESULT IS` checks the answer the rule returned.

`WAS CALLED WITH ARGS(...)` checks that a particular thing happened, to a particular claim. Naming the claim matters more than it looks: a rule that approves *something* is not the same as a rule that approves *this*, and on a rule handling two claims the difference is the whole test.

`WAS NOT CALLED` checks that something did not happen. This is the half people leave out, and it is often the more valuable one — a rule that approves and *also* refuses has a bug that no amount of checking the approval would find.

When it passes

```
3/3 passed
PASSED UnderTheLimitIsApproved
PASSED OverTheLimitIsRefused
PASSED ExactlyAtTheLimitIsApproved
```

Three tests of the same rule: under the limit, over it, and exactly on it.

That third one is there because somebody had to decide. Is a claim of exactly 200.00 against a limit of 200.00 approved? The rule says `IF total > limit THEN` refuse, so it is approved — but the policy document might have meant otherwise, and until somebody writes the case down, nobody knows whether the rule matches the policy or merely matches itself.

When it fails

Suppose somebody believed the limit was meant to be exclusive, and wrote the case that way:

```
0/1 passed
FAILED TheLimitIsInclusive
line 11: expected the result to be false, but it was true
        RESULT IS FALSE
```

The report names the test, the line, what was expected and what happened. Nothing about stack frames or which framework noticed.

And note what this failure actually is. Neither the rule nor the test is broken; they disagree about policy. The rule treats the limit as inclusive, the test expects exclusive, and the failure is the disagreement surfacing where somebody can settle it. That is the most useful kind of test failure there is, and it is only available because both sides are readable by the person who can settle it.

Enough to start

You can now write tests for rules you already have: name the case, answer the questions the rule asks, give it its arguments, run it, and say what should be true.

What you cannot yet do is describe a world more complicated than one claim and one answer — several calls to the same operation, arguments you only partly care about, operations that write values back. The rest of this part is those.

CHAPTER 14

Standing in for the world

Your rule calls operations that load claims, send messages and charge budgets, and a test cannot have those really happen. Mocking as an idea: the rule under test stays real, everything it reaches for is pretended. What that buys, and the one thing it can never tell you.

The problem, stated plainly

A rule is a small thing surrounded by a large world. `APPROVE` moves money. `ROUTE` reads the approval policy out of some system. `ANOMALY_SCORE_OF` asks an LLM. Running a rule for real means running all of that for real, which is not a test — it is a payment.

So a test replaces them. The rule itself stays exactly as it is, unmodified and uninstrumented, and every operation it reaches for is stood in for by something the test controls.

Two words for the same idea: the operations are **mocked**, and the rule is the **subject**.

What that gets you

Determinism. The rule asked what the claim came to and got 128.40, because the test said so. Not because a database happened to hold that today.

Cases you could not otherwise arrange. A claim that breaches the budget of a cost centre that does not exist. A model that returns 10. A claim with no lines at all. Every one of these is one line in a test and a week of arranging in a real system.

Speed, and no consequences. Nobody is paid, nothing is written, nothing is sent.

A record of what the rule did. The framework remembers every call — which operations, in what order, with what arguments — so a test can assert on things that leave no return value. `APPROVE` answers nothing; the only evidence it

ran is that it was called.

What it can never tell you

One limitation, and it is not small, so it belongs here rather than in a footnote.

A mocked operation tells you nothing about the real one. If a test says `TOTAL_OF` returns 128.40 and the real `TOTAL_OF` has a rounding bug, every test in your suite passes and the rule is wrong in production. Mocking tests the *rule*, on the assumption that the vocabulary does what its description says.

That assumption is the seam. It is not a flaw in the approach — it is the same seam that lets a subject-matter expert review the rule without reading Java, and you cannot have one without the other. But it means a BUNIT suite is not the only testing your application needs. The operations themselves are ordinary Java and need ordinary Java tests, which is Part 3's problem.

What you get is a clean division: BUNIT proves the policy is encoded correctly, and Java tests prove the operations do what they claim. Neither substitutes for the other.

Mocking is not optional here

In most codebases, mocking is a matter of taste. You can argue for a real database in tests, and plenty of people do, with reason.

That argument does not survive contact with a BUBAS vocabulary, for a structural reason worth seeing. Every value the domain owns is opaque — a `Report`, an `Item`, a `CostCentre`. A test cannot construct one, because there is no syntax for constructing one and no way to reach inside if there were. So a test cannot hand a rule a real claim. It never had that option.

What it can do is hand the rule something that *stands for* a claim, and mock every operation that would have looked at it. That is the next chapter.

The consequence is worth stating: **an opaque-valued surface has to be mocked as a whole.** If `ITEM_AT` is mocked and returns a stand-in, then `AMOUNT_OF` and `CATEGORY_OF` must be mocked too, because the real ones expect a real line and will get something else. Leaving half of it real is the

commonest mistake in a first BUNIT test, and chapter 17 is about where the line honestly falls.

What it looks like

Three shapes cover nearly everything.

An operation that answers. `"TOTAL_OF" WITH ARGS("the claim") RETURNS 128.40` — when asked about that claim, say this. Without `WITH ARGS`, it answers the same for any call.

An operation that does something. `"APPROVE _" IS MOCKED` — let it be called, do nothing, remember that it happened. There is nothing to return, so there is nothing to say.

An operation that writes into a variable. `"ROUTE _ TO _ AT _" SETS "approver" TO "..."` — because a mocked command does not run its own handler, so whatever it would have written has to come from somewhere. Chapter 18 is about the checking around that, which is stricter than you expect and for good reason.

The thing to keep hold of

A test describes a world and then asserts what the rule did in it. The mocks are the world. They are not the thing being tested, and every line of them is an assumption you are making about the vocabulary.

That is worth remembering when a test grows long. Twenty lines of mocks before a two-line assertion usually means the rule reaches for too much, and the rule is what you should look at — not the test.

CHAPTER 15

Tokens

Standing in for opaque values you cannot construct; naming rather than building.

A value a test cannot make

The rule from chapter 2 needs a claim. A test has no way to make one.

That is not an oversight. Chapter 5 established that a domain value is sealed: a program can hold one, pass it, and ask questions about it, and there is no syntax anywhere in BUBAS for building one or looking inside. A test is a BUBAS program, so a test cannot build one either.

The way out is to notice what a rule can actually observe about a claim. It can pass it to operations, and it can tell whether two of them are the same one. That is the entire list. Anything else it might want to know is a question it has to ask, and in a test the answer to every such question comes from a mock.

So the claim itself never needs to exist. It needs a **name**.

```
ARGUMENT "claim" IS "Alice's claim"
```

`"Alice's claim"` is a token: a stand-in for a claim, identified by what the test called it. There is no claim anywhere. There is a label, and everything the rule does with it is arranged by the test.

The rule that makes it work

A string written where an opaque value belongs names a token.

That is the whole rule, and it holds everywhere: in `ARGUMENT`, in what a mock returns, in what a mock matches on, in what an expectation checks. It is unambiguous because opaque values are the one kind BUBAS cannot construct, so a string in that position could not have meant anything else.

```
"TOTAL_OF" WITH ARGS("Alice's claim") RETURNS 128.40
```

Both halves of that line use it. The argument matched on is a token — *when asked about the claim called this* — and if `TOTAL_OF` returned a claim rather than a decimal, the answer would be one too.

Tokens flow

A token handed to a rule comes back out the other side, and the framework keeps track:

```
"APPROVE _" WAS CALLED WITH ARGS("Alice's claim")
```

That is not merely checking that something was approved. It checks that *this* claim was — the one the test named and handed in. The rule passed it through `TOTAL_OF`, past a comparison and into `APPROVE`, and it arrived as the same thing it started as.

Two claims of the same kind

The interesting case is more than one. A rule that compares two claims and acts on one of them has a bug worth catching: acting on the wrong one.

```
PROGRAM TwoClaimsToldApart
    "TOTAL_OF" WITH ARGS("the small claim") RETURNS 40.00
    "TOTAL_OF" WITH ARGS("the big claim")   RETURNS 900.00
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "the big claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS FALSE
    "REJECT _, _" WAS CALLED WITH ARGS("the big claim", ANYTHING())
    "TOTAL_OF" WAS CALLED WITH ARGS("the big claim")
END.
```

```
1/1 passed
PASSED TwoClaimsToldApart
```

Two tokens of the same type, coexisting. The mocks answer differently for each, and the expectation names which one was refused. Nothing here depends

on the order the rule happened to ask in — the identification is by name, so a rule that asked about the small claim first would still be tested correctly.

Names are for reading

A token's name has no meaning to anything. `"o1"` would work as well as `"Alice's claim"`.

Use the second. The name appears in the call log and in every failure message, and the difference between

```
line 13: expected REJECT _, _ to be called with ("the small claim",
■ anything), but it was called with ("the big claim", "over the
■ 200.00 limit")
        "REJECT _, _" WAS CALLED WITH ARGS("the small claim",
        ■ ANYTHING())
```

The same sentence with `o1` and `o2` in it is a puzzle rather than a diagnosis, and the good names cost nothing.

Where it stops

Two limits worth knowing before you meet them.

A token that reaches a real handler fails. A token is not an instance of the Java class behind the type, so an operation you left unmocked will be handed something it cannot use. The error says so, but the fix is the previous chapter's rule: an opaque-valued surface is mocked as a whole.

Some tokens the framework supplies for you. When a mocked command writes into an opaque variable — `ROUTE claim TO approver AT centre` — nothing in the test needs to name the cost centre, because a token is the only thing it could be, and the framework provides one. You will see it in the call log as `"#1"`. What a command writes into a *non*-opaque variable is a different matter, and the next chapters take it up.

CHAPTER 16

Matching arguments

Saying which calls a mock should answer: exact values, ranges, anything at all, anything but. How much to pin down, and why pinning down everything produces a test that fails whenever anyone touches anything.

Two places arguments are matched

Arguments come up twice in a test, and it is worth keeping them apart.

On the way in, a mock decides whether a call is the one it was set up for:

```
"TOTAL_OF" WITH ARGS("the big claim")   RETURNS 900.00
```

On the way out, an expectation checks a call that happened:

```
"REJECT _, _" WAS CALLED WITH ARGS("the big claim", ANYTHING())
```

The same matching applies in both directions, which is deliberate — a test that could describe a call one way and check it another would eventually disagree with itself.

Exact values

Write a value and it must match. Text matches text, numbers match numbers, a name matches the token it names.

`DECIMAL` compares by value rather than by how it was written, exactly as `=` does in the language, so a mock answering `1.50` satisfies an expectation of `1.5`. That is the same rule chapter 3 gave, applied in the one place where it would otherwise be surprising.

Saying you do not care

Most calls have an argument the test has no opinion about. `ANYTHING()` says so:

```
PROGRAM WhatMattersAboutTheRefusal
    "TOTAL_OF" WITH ARGS(ANYTHING()) RETURNS 1230.00
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "some claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS FALSE
    "REJECT _, _" WAS CALLED WITH ARGS(ANYTHING(), STARTS_WITH("over
    ■ the"))
    "REJECT _, _" WAS CALLED WITH ARGS(ANYTHING(), CONTAINS("200.00"))
END.
```

```
1/1 passed
PASSED WhatMattersAboutTheRefusal
```

This test is about the *reason* a claim was refused, not about which claim it was. So the claim is `ANYTHING()` in both the mock and the expectation, and the reason is checked two ways.

Matching text

Refusal reasons are text built out of values, and pinning the whole string is usually the wrong move. Three matchers cover most cases:

- `CONTAINS("200.00")` — the limit appears in the message somewhere
- `STARTS_WITH("over the")` — it begins the way the policy says it should
- `ENDS_WITH(...)`, and `MATCHES(...)` for a pattern when nothing simpler will do

The choice is a judgement about what the message is *for*. If the claimant is shown it and the policy says the amount must appear, then `CONTAINS("200.00")` is the requirement, written down. Pinning the entire sentence tests the wording, which nobody promised to keep.

Matching numbers

`GREATER_THAN`, `LESS_THAN`, `BETWEEN`, `AT_LEAST`, `AT_MOST` do what they say, and are most useful where a rule computes a figure your test should not have to

reproduce. Asserting that an escalation happened with a remaining budget `BETWEEN(2000.00, 3000.00)` says what you meant; asserting exactly 2500.00 says that plus an arithmetic claim you did not intend to make.

`ANYTHING_BUT(...)` is the odd one out and earns its keep: `ANYTHING_BUT("")` catches an empty reason, which is the failure mode a message-building bug actually has.

How much to pin down

This is the part that decides whether a suite is an asset or a liability, so it deserves a rule.

Pin what the policy says. Leave everything else alone.

A test that pins every argument of every call is a test that fails when somebody reorders two harmless lines, rewords a message, or adds an argument nobody reads. Each of those failures costs somebody an hour and teaches them that the suite cries wolf. After enough of them, people stop reading failures and start re-running builds.

A test that pins too little passes when the rule is wrong, which is worse but at least obvious once you look.

The way through is to ask, for each argument: *if this changed, would the policy have changed?* If yes, pin it. If no, `ANYTHING()`.

One thing you cannot say

There is `WAS CALLED`, `WAS NOT CALLED`, and `WAS CALLED WITH ARGS(...)`.

There is no `WAS NOT CALLED WITH ARGS(...)`. You can assert that an operation was never called at all, and you can assert that it was called with particular arguments, but not that it was never called with a particular set. If your test needs that — *the small claim must never have been refused* — it has to be arranged some other way, usually by writing a case in which refusing it would change the result.

Worth knowing before you write the line and find it does not parse.

CHAPTER 17

Leaving some of it real

Not everything needs standing in for. Partial mocking, when leaving an operation real makes a test more honest, and when it quietly hides the thing you meant to check.

Nothing says you must mock everything

A test mocks what it needs to control. Operations it says nothing about run for real.

Here is the rule from chapter 4 — the one that records notes as it decides — tested without mocking `NOTE`:

```
PROGRAM NotesAreLeftReal
    "TOTAL_OF" WITH ARGS("Bob's claim") RETURNS 1230.00
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "Bob's claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS FALSE
    "REJECT _, _" WAS CALLED
END.
```

`TOTAL_OF`, `APPROVE` and `REJECT` are stood in for. `NOTE` is not mentioned, so the real one runs, and what it wrote is there afterwards:

```
1/1 passed
PASSED NotesAreLeftReal

what the rule wrote:
NOTE: checked against a limit of 200.00
NOTE: over by 1030.00
```

The second line is arithmetic the rule did — 1230.00 over a limit of 200.00 — and no mock supplied it. Had `NOTE` been mocked, that line would have vanished and the test would have been blind to a rule that computed the overage

wrongly.

When leaving it real is right

When the operation is the thing you want to observe. A rule's notes and messages are often the only externally visible evidence that it reasoned correctly. Mocking the operation that records them throws that evidence away.

When it has no side effects worth avoiding. `NOTE` writes to a log the test can read. Nothing is paid, nothing is sent, nothing is stored. There is no reason to replace it.

When it is pure arithmetic over values the test already controls. An operation that formats a message or converts a currency, given inputs the mocks supplied, adds no unpredictability.

When it is not

When it touches the world. `APPROVE` pays. `ROUTE` reads the approval policy from somewhere. Anything that writes, sends, charges or asks a service gets mocked, and this is not a judgement call.

When it will not answer the same way twice. Chapter 19's operation is the extreme case.

When it needs a real domain value. This is the one that catches people, and it follows from chapter 15. A token is not a real claim, so an operation you left real will be handed something it cannot use and will fail — not with a helpful message about mocking, but with a type error from a handler.

The rule that follows is worth memorising: **an opaque-valued surface has to be mocked as a whole.** If `ITEM_AT` is mocked and hands back a stand-in line, then `AMOUNT_OF`, `CATEGORY_OF`, `MERCHANT_OF` and `HAS_RECEIPT` must be mocked too. Mocking half of it is the commonest mistake in a first BUNIT test, and the framework goes out of its way to explain it when it happens.

The honest question

There is a temptation to leave things real because mocking them is tedious, and to tell yourself the test is more realistic for it. Sometimes that is true. Often it is the beginning of a test that passes for reasons nobody understands.

The question to ask is: **is this operation part of what I am testing, or part of the world I am describing?**

The rule under test is what you are testing. Everything it reaches for is the world. An operation left real is a piece of the world you have decided not to control — which is fine when its behaviour is obvious and its output is something you want to see, and a slow leak when it is neither.

A middle case worth naming

Some operations sit genuinely on the line. An operation that formats a claimant-facing message from values your mocks supplied is arithmetic, and leaving it real tests it for free — but it is also somebody else's code that could change under you, and a test that starts failing because a message was reworded is the false alarm chapter 16 warned about.

The resolution is usually the same as there: leave it real, and assert on the part of the output the policy actually requires. `CONTAINS("200.00")` survives a rewording; the whole sentence does not.

CHAPTER 18

Mocks that cannot be right

A mock that sets no value the rule then reads, or answers a call the rule never makes, is not a test — it is a test-shaped object that passes. What the consistency checker refuses, and why it is deliberately narrow: a check that fires too widely destroys the thing it was protecting.

A test that passes and proves nothing

The worst outcome in testing is not a failing test. It is a test that passes without exercising anything — green, reassuring, and empty.

BUNIT refuses a family of those before the rule ever runs. This chapter is what it refuses, and more usefully, why the list stops where it does.

A command that writes

Chapter 4 said some operations do rather than answer. A few do both: they perform something *and* put a value into a variable for the rule to use afterwards.

The routing rule from stage 6 uses one. `ROUTE claim TO approver AT centre` reads the approval policy once and produces two things — who has to sign, and which budget it charges — because those are decided together and asking twice could get answers from two different readings.

Here is a test of it, and the calls it produced:

```
PROGRAM OverLimitGoesToAManager
    "TOTAL_OF" WITH ARGS("Erin's claim") RETURNS 450.00
    "ROUTE _ TO _ AT _" IS MOCKED
    "ROUTE _ TO _ AT _" SETS "approver" TO "the line manager"
    "BUDGET_LEFT" RETURNS 2500.00
    "ESCALATE _, _" IS MOCKED
    "APPROVE _" IS MOCKED
    "REJECT _, _" IS MOCKED

    ARGUMENT "claim" IS "Erin's claim"
    ARGUMENT "limit" IS 200.00

    RUN

    RESULT IS FALSE
    "ESCALATE _, _" WAS CALLED
    "APPROVE _" WAS NOT CALLED
END.
```

```
1/1 passed
PASSED OverLimitGoesToAManager

what the rule was given:
TOTAL_OF("Erin's claim")
ROUTE _ TO _ AT _("Erin's claim", "the line manager", "#1")
BUDGET_LEFT("#1")
ESCALATE _, _("Erin's claim", "needs the line manager, 2500.00 still
■ available")
```

Two things in that call log are worth slowing down for.

The approver reads `"the line manager"` — the test supplied it, with `SETS`. A mocked command does not run its own handler, so nothing else could have written it.

The cost centre reads `"#1"`. Nobody wrote that. It is a token the framework supplied, because `centre` is an opaque variable and a token is the only thing it could possibly be — the rule cannot look inside it, and every operation that would have is mocked anyway. So the test says nothing about it, and one flows on into `BUDGET_LEFT`.

That is the rule, and it is worth stating as one: an opaque target is filled in for you; anything else you must supply.

What happens if you forget

Take the `SETS` line away and the test is refused:

```
line 10: 'ROUTE _ TO _ AT _' is mocked, so its handler will not write
■ 'approver' — and nothing supplies it on every path. Add: "ROUTE _
■ TO _ AT _" SETS "approver" TO ...
        RUN
```

Notice three things about that message.

It names **what** is unset, **why** nothing will set it — the command is mocked, so its handler will not run — and **what to write instead**, in the syntax you need. It also fires at `RUN`, not at the expectation that eventually reads a garbage value, which is where the confusion would have been.

And notice what would have happened without the check. The rule would have read an unset variable, and either failed with something baffling or, worse, worked by accident and left you with a passing test of nothing.

The other refusals

The same checker enforces a handful of related things, all with the same shape: *this test cannot be meaningful, so it will not be run.*

An expectation before the run. Asking what a rule did before `RUN` is asking about something that has not happened. The message says so.

An expectation on an operation that was never mocked. You cannot ask whether `APPROVE` was called if nothing was watching. Declaring it mocked is what starts the watching.

A mocked command declared but never reachable. A mock for something the rule cannot call on any path is either a typo or a leftover from a rule that changed. Both are worth knowing about.

`SETS` **on a command that is not mocked.** Then the real handler writes the variable, and what you supplied would be silently ignored. The message says exactly that: *supplying it would be ignored.*

Why it stops there

The checker could do much more, and deliberately does not.

It could complain about a mock whose return value is never used. It could insist every mocked operation is asserted on. It could require a `RESULT IS` in every test. Each of those catches a real mistake sometimes, and each of them fires on perfectly good tests the rest of the time.

That trade is worse than it looks, because of what happens to a check that cries wolf. People suppress it, or work around it, or stop reading its output — and then it is not protecting anything while still costing everyone time. **A check that fires too widely destroys the thing it was protecting.**

So the line is drawn at tests that *cannot* be meaningful, not tests that *look* careless. Every refusal above is a case where the test would have been checking nothing, and the framework can prove it. Where it cannot prove it, it says nothing and leaves the judgement to you.

A useful side effect

Because the checker traces every path through a test before running it, it is also flow-sensitive in a way you can lean on. A test can branch and loop like any BUBAS program, and the checker follows — it will not accept a run that only happens on one path, or a value supplied inside an `IF` that the rule might read outside it.

There is a pleasing consequence. A loop that drives several runs must be one whose body is guaranteed to execute, for exactly the reason chapter 8 gave about `lastMerchant`: a top-tested loop might run zero times, and the checker will not assume otherwise. The bottom-tested form works. Nobody coordinated that; the checker and the language reached the same conclusion from the same reasoning.

CHAPTER 19

Testing what cannot be tested

The model-backed operation from chapter 10 will not give the same answer twice, so there is no version of this test that does not stand in for it. Mocking stops being good practice here and becomes the only way the rule can be checked at all — and the rule around it, the threshold you chose, is what you are really testing.

The wall

Chapter 10 added an operation that asks a model how unusual a line of spending looks. Everything about writing rules with it was straightforward. Testing one is not.

Take the obvious test: a claim with a large unreceipted dinner should be escalated. Run it. It escalates. Run it next month, after somebody retrained the model, and it does not. Neither run tells you anything about your rule, because the input to the decision changed underneath it.

You cannot assert the claim is escalated. You cannot assert it is *not*. You cannot even assert the test is stable enough to run twice.

This is the wall every argument for mocking eventually gestures at and rarely reaches. Nobody mocks a database because they must — you can run a database. Here there is no version of the test that works by running the real thing.

What you are actually testing

Once you accept that the score has to be pinned, something clarifies.

You were never testing the model. You have no way to test the model from here, and it is not yours to test. What you are testing is **the rule you wrapped around it**: that a score of 8 escalates when the threshold is 8, that the worst line wins rather than the first, that a claim which trips nothing still gets checked against the limit.

Those are policy questions with right answers, decided by a person, and they are exactly what the person who owns the policy would want checked.

```
PROGRAM AFlaggedLineIsEscalated
    "ITEM_COUNT" WITH ARGS("the claim") RETURNS 2
    "ITEM_AT" WITH ARGS("the claim", 1) RETURNS "the taxi"
    "ITEM_AT" WITH ARGS("the claim", 2) RETURNS "the dinner"
    "ANOMALY_SCORE_OF" WITH ARGS("the taxi")   RETURNS 2
    "ANOMALY_SCORE_OF" WITH ARGS("the dinner") RETURNS 8
    "TOTAL_OF" WITH ARGS("the claim") RETURNS 411.00
    "ESCALATE _, _" IS MOCKED
    "APPROVE _" IS MOCKED

    ARGUMENT "claim"  IS "the claim"
    ARGUMENT "limit"  IS 2000.00
    ARGUMENT "flagAt" IS 8

    RUN

    RESULT IS FALSE
    "ESCALATE _, _" WAS CALLED WITH ARGS("the claim",
    ■ CONTAINS("scored 8"))
    "APPROVE _" WAS NOT CALLED
END.
```

Two lines, two scores, one over the threshold. The rule should escalate and should not approve.

The threshold is the thing

Here is the same test with one number changed — the threshold, not the score:

```
ARGUMENT "flagAt" IS 9

RUN
```

The dinner still scores 8. The model's opinion has not changed. The claim is approved, because the rule now flags at 9.

```
2/2 passed
PASSED AFlaggedLineIsEscalated
PASSED TheThresholdIsTheRule
```

Two tests, both passing, differing in one number that a person chose and can be asked about. That pair is the argument of chapter 10 turned into something a

build can check: the policy is in the rule, and moving it changes outcomes in a way somebody is accountable for.

Pinning a score is a claim about the world

One honest complication. When a test says the dinner scores 8, it is asserting something about a model it cannot verify.

If the real model never returns 8 for anything resembling that line, the test is checking a case that does not occur. It will pass forever and protect nothing — a subtler version of the test-shaped object chapter 18 was about, and one no checker can catch.

There is no clean fix, only a practice. **Pin scores at the boundaries your rule cares about, not at arbitrary numbers.** A rule that flags at 8 should be tested at 7, 8 and 9, because those are the values where its behaviour changes. Whether the model ever produces 8 for a given line matters much less than whether your rule does the right thing when it does.

That also keeps the tests useful when the model is replaced. The boundaries are yours; the distribution is not.

What the tests cannot tell you

Worth stating plainly, because a green suite is persuasive.

Not whether the scores are any good. A model that scores everything 5 would pass every test here.

Not whether the threshold is right. The tests prove the rule does what it says at 8. Whether 8 is the correct place to draw the line is a policy question, decided by a person, and no test decides it. What the tests do is make the decision visible and its consequences checkable — which is all this book ever claimed for the arrangement.

Not whether the operation is safe. The vocabulary bounds what a rule can *name*, not what a named thing *does*, and an operation backed by a service is where that gap is the widest. Chapter 32 takes it up.

The shape this leaves you with

A rule that consults a model ends up with more tests than a rule that does not, and they are the straightforward kind: for each threshold, a case just under, a case exactly on it, and a case just over.

That is a good outcome. The unpredictable part has been pushed behind a boundary, and what is left in front of it is a policy you can enumerate. The alternative — a rule where the model decides — would have nothing to enumerate at all, which is chapter 25's argument seen from the testing side.

CHAPTER 20

A suite that stays honest

Organising tests so they keep their value: what to cover, what not to bother with, and the coverage claim a finite vocabulary makes possible that a general-purpose language cannot — you can enumerate every operation a rule is able to call.

The claim a small language lets you make

Ask what a piece of Java can do and there is no answer. It can call anything on the classpath, and anything those call, indefinitely. Coverage is measured in lines executed because the set of *things reachable* has no edge to measure against.

A BUBAS rule is different. Every operation it can call is in the vocabulary, and the vocabulary is a list somebody wrote. So a question that is unanswerable elsewhere is arithmetic here:

Which operations can this rule call, and which does my suite exercise?

Both sides are finite and both are enumerable. That is not a coverage percentage; it is a complete account, and it is worth building a suite around.

What to cover

Three things, in order of how much they repay.

Every branch out of the rule. If it can approve, refuse or escalate, there is a test for each. This is the minimum, and it is where a suite starts.

Every threshold, three times. Just under, exactly on, just over. The exactly-on case is the one that finds real disagreements, because it is where the policy document and the rule most often part company — chapter 13's inclusive-limit example is exactly this. If a rule has three thresholds, that is nine tests, and they are cheap ones.

Every operation the rule calls, in at least one test that leaves it real or checks its arguments. Not to test the operation, which is Java's job, but to notice when the rule stops calling something it used to.

What not to bother with

Equally important, and less often said.

Combinations for their own sake. Three thresholds do not need eight tests of every combination. They need nine tests of the boundaries and one or two of realistic claims. Combinatorial suites grow faster than the understanding they provide.

The same case twice in different words. Two tests that differ only in the claim's name are one test.

Wording. Chapter 16's rule again: pin what the policy requires, not the sentence.

Anything the compiler already refuses. There is no value in a test asserting that a variable must be assigned before use, or that a claim cannot be compared to another. Chapter 8's whole point is that those are not runtime concerns. A suite that tests them is testing BUBAS, which somebody else already did.

Naming

The report is read by people who did not write the tests, and it prints names:

```
3/3 passed
PASSED UnderTheLimitIsApproved
PASSED OverTheLimitIsRefused
PASSED ExactlyAtTheLimitIsApproved
```

`UnderTheLimitIsApproved` says what case failed. `TestApproveExpense3` says a number. Name the case in the words the policy uses, and a failing build tells somebody which rule is in dispute before they open anything.

One file per case

Keep each test in its own file, named after the case. It costs nothing, and it means a failure names a file somebody can open, a rule change touches only the cases it affects, and two people can add cases without meeting in the same file.

The alternative — a few large files grouping many cases — saves a little typing and costs the thing that makes this whole arrangement work: that a subject-matter expert can be pointed at *one* file and asked whether it is right.

Keeping it honest over time

Suites rot in predictable ways, and two are worth guarding against deliberately.

Tests that stopped meaning anything. A rule changes, a test still passes because its mocks happen to satisfy the new shape, and nobody notices it now exercises nothing. Chapter 18's checker catches the mechanical cases. The rest is caught by the habit of deleting: when a rule changes, read its tests and remove the ones that no longer describe a case anybody cares about. A suite you prune is a suite people trust.

Mocks that drifted from the operations. Every mock is an assumption about what an operation does. When the vocabulary changes — an operation's meaning shifts, a return type narrows — the mocks do not automatically notice. The vocabulary document from chapter 11 is the thing to reread when that happens, and the review checksum in chapter 26 is what tells you it happened at all.

The end of Part 2

You can now write a rule, and check it, in a language the person who owns the rule can read. Both artefacts — the policy and the evidence that it works — are reviewable by the same person, which is the whole of what chapter 1 said was missing.

What has been assumed throughout both parts is that somebody built the vocabulary. Every operation in these examples exists because a person decided this domain has that question and made it askable, wrote its description, decided whether it returns a score or a verdict, and chose whether it answers or does. Those decisions have shaped every page so far.

Part 3 is where they are made.

PART 3

Embedding

CHAPTER 21

Three objects

The whole architecture in one chapter: a language that is defined once and sealed, a program that is compiled once and reused, and an interpreter that is cheap, single-use and single-threaded. What each costs, what each is safe to share, and why sealing exists.

Welcome to the Java

Parts 1 and 2 contained none, deliberately: everything up to here can be read by the person who owns the rules, and defining a vocabulary is a different job. This is that job.

It rests on three objects, and almost every question about embedding BUBAS is a question about which of the three you are holding.

Object	Made	Cost	Shared
`BubasLanguage`	once, at startup	expensive	freely, across threads
`BubasProgram`	once per rule text	moderate	freely, across threads
`Interpreter`	once per execution	trivial	never

One whole example first

Everything below is one embedding, small enough to read at once. It has a single type, a single question, a single instruction — and it is complete: there is nothing left out to make it fit on the page.

The language

A `BubasLanguage` is the vocabulary: every type, function and statement a program written against it may use. It is built with a builder and then **sealed**.

```
/** Built once, at startup, and held for the life of the process. */
static final BubasLanguage LANGUAGE = BubasLanguage.builder()
        .install(Standard::register)
        .defineOpaqueType("Claim", Claim.class)
        .defineFunction("TOTAL_OF", TotalOf.class)
        .defineStatement("APPROVE {expression/Claim:expense}",
        ■ Approve.class)
        .seal();
```

`install(Standard::register)` brings in declaration and assignment. Chapter 4 made the point from the other side: those are ordinary statements shipped in a module, and an embedder who wants different ones simply does not install these.

Everything after it is this application's own. `Claim` is a Java class of yours; `TOTAL_OF` and `APPROVE` are Java classes of yours. The next three chapters are about writing them.

Build this once, in startup, and hold it for the life of the process. It is immutable and thread-safe once sealed.

Sealing

`seal()` is where the language stops being editable and starts being usable. It is not a formality.

At `seal()` the analysis runs that proves **no two statement patterns can ever match the same line**. Chapter 24 covers what that means and how it can fail; what matters here is when it happens. A vocabulary with an ambiguity in it fails at startup, with a message naming both patterns — not on the first unlucky program six weeks later.

That is the general shape of the design: expensive checks run once, at the moment the vocabulary is fixed, so that everything downstream is cheap and certain.

The program

This is the rule the example runs — an ordinary text file, kept wherever your application keeps them:

```
PROGRAM ApproveSmallClaim(expense Claim, limit DECIMAL) RETURNS
■ BOOLEAN
    DECLARE total DECIMAL

    total = TOTAL_OF(expense)

    IF total > limit THEN
        RETURN FALSE
    END IF

    APPROVE expense
    RETURN TRUE
END
```

```
/** Compiled once per rule text, then reused for every claim that
■ arrives. */
static final BubasProgram PROGRAM =
■ LANGUAGE.compile(Runs.source("first-rule.bu"));
```

`Runs.source` there is nothing but a file read, spelt however suits you. `compile` produces a `BubasProgram`: a rule that has been lexed, parsed, matched against the vocabulary, type-checked and flow-analysed. Everything chapter 8 listed has already happened by the time you hold one.

A `BubasProgram` is immutable and thread-safe, and it carries the language it was compiled against. Compile a rule once and reuse it for every claim that arrives — compiling per request works, and is simply waste.

Compilation is where a bad rule is rejected. A `BubasException` from `compile` carries the line number, the source line, and a diagnostic. Chapter 30 is about who should see it.

The interpreter

```
/** One interpreter per decision. Cheap to make, used once, thrown
■ away. */
static boolean decide(Claim claim, BigDecimal limit) {
    return Interpreter.of(PROGRAM)
            .argument("expense", claim)
            .argument("limit", limit)
            .run()
            .asBoolean();
}
```

An `Interpreter` holds the state of one execution: the variables, the arguments, and whatever the application supplies for this run. It is deliberately cheap to make, because you make one per claim.

Three rules, and they are absolute.

One interpreter, one run. It is not reusable. Make another.

One interpreter, one thread. There is no locking in it, because it never needs any.

Nothing is shared between runs. Two claims decided at the same moment cannot see each other's variables, because they are in different objects. A BUBAS program has one global scope and no concurrency *within* a program, which means concurrency *between* programs costs nothing to reason about.

Note what `argument` is doing. It checks the value against the parameter's declared type immediately, so a wiring mistake surfaces before the rule runs rather than at the first use.

Why it is three and not one

The obvious design is one object: hand it a source string and some arguments, get an answer. It would be a smaller API and a worse one.

Splitting them puts each cost where it belongs. The expensive work — the overlap analysis, the type checking, the flow analysis — happens once per vocabulary and once per rule. The per-claim work is allocating a small object and walking a tree.

It also puts each *failure* where it belongs. A vocabulary that is ambiguous fails at startup. A rule that does not compile fails when it is saved, in front of whoever wrote it. A rule that divides by zero fails on the claim that did it. Three failure modes, three moments, three audiences — chapter 30's subject.

And it makes the sharing rules simple enough to hold in your head, which the table at the top is.

What is coming

The next three chapters are the vocabulary itself: types, functions, and the pattern language that makes statements. They are the design work that decides how good Parts 1 and 2 feel to the people living in them.

CHAPTER 22

Defining types

Opaque types from the embedder's side; total opacity as a design choice.

A Java class becomes a domain value

```
/** A value BUBAS holds and passes but cannot look inside. */
@BubasDescription("""
        One employee's expense claim for a trip or a period.
        A program is given one to decide about; ask TOTAL_OF what it
        ■ comes to.
        """)
public static final class Report {
    final long id;
    private final String employee;
    private final LocalDate submitted;
    private final List<Item> items;

    Report(long id, String employee, LocalDate submitted, List<Item>
    ■ items) {
        this.id = id;
        this.employee = employee;
        this.submitted = submitted;
        this.items = items;
    }

    BigDecimal total() {
        return items.stream().map(Item::amount).reduce(BigDecimal.ZERO
        ■ , BigDecimal::add);
    }

    @Override
    public String toString() {
        return "report " + id + " (" + employee + ")";
    }
}
```

An ordinary class. It has fields, methods, a `toString`, and none of that is visible to BUBAS. It implements no interface, extends nothing, and touches nothing from the BUBAS API.

The one concession is the description, which says what a claim is in the business rather than what it holds. It is read when the vocabulary is exported for review — chapter 26 — and ignored the rest of the time.

It becomes a type by being registered:

```
.defineOpaqueType("Report", Report.class)
```

The name is what programs write. The Java class is what handlers receive. Nothing connects them except this line, which means the same class can be exposed under different names in different languages, and a language can be built over classes you do not own.

The name is a reserved word

Registering `Report` reserves it — in every casing. `report`, `REPORT` and `rEpOrT` are all taken, which is why every program in this book calls its variable `claim`.

That surprises people, so it is worth deciding names with it in mind. A type named after the thing its variables will want to be called is a type whose users have to think of a synonym every time. `Report` and `claim` work because they are different words for the same idea; `Claim` and `claim` would not.

The diagnostic says what happened rather than merely reporting a reserved word, which helps, but not as much as picking the name well.

Opacity is total, and that is the design

A registered type is a black box. Programs cannot read fields, call methods, render one as text, compare two, or construct one. Chapter 5 covered what that feels like from inside a rule. Here is why you should want it.

The vocabulary stays a list somebody wrote. If rules could read fields, then what a rule can know would be decided by the shape of a Java class — and classes grow. Somebody adds a field for an unrelated feature and the vocabulary has silently gained a word nobody decided on, documented, or reviewed. Chapter 11's document would stop being a complete account.

Your domain model stays yours. Because `Report` is opaque, you can rename its fields, change its representation, or replace it with a different class entirely, and no rule breaks. The only contract is the operations, which is a contract you wrote deliberately. Partial exposure would make every field a public API you did not know you had published.

Tests become possible. Chapter 15's tokens work precisely because a rule cannot look inside. A partially transparent value could not be stood in for by a name.

That third one is worth dwelling on. Total opacity is not one design decision; it is what makes the testing story in Part 2 available at all. A vocabulary that exposed a claim's total as a readable field would have saved one operation and cost the whole of BUNIT.

Describing it, and the one case that cannot

The description goes on the class, and `defineOpaqueType` takes it from there. Nothing else is needed, and nothing else is registered.

That is the ordinary case because it is the common one: a class your own team wrote, for a domain your own team models, exposed to a language your own team built. Requiring anything more of it would make the common case pay for the rare one.

The rare one is real, though. Some types cannot carry the annotation:

- a class from a library — a `java.time.LocalDate`, a `BigDecimal` wrapper somebody else ships
- a domain model shared with consumers that must not depend on BUBAS, where adding the annotation would make your model depend on one of the things that reads it
- a generated class, where the next generation would drop it

For those, describe the type on an empty interface and register it with `defineOpaqueTypeVia`. This vocabulary has exactly one, because a claim has a date on it and nobody can annotate `java.time.LocalDate`:

```
/**
 * The escape route, and the case it exists for: nobody can put an
 ■ annotation on
 * {@code java.time.LocalDate}. An empty interface stands in for it
 ■ and carries the words.
 */
@BubasDescribes(LocalDate.class)
@BubasDescription("""
        A calendar day, with no time of day and no timezone.
        Ask DAYS_BETWEEN how far apart two of them are.
        """)
public interface DateDoc {
}
```

`@BubasDescribes` says which class the interface stands in for; the builder reads the class from it, so the class is never named at the registration:

```
/**
 * Stage 8: a type from the standard library, described through an
 ■ interface because the class
 * itself cannot be annotated. Everything else here is described on
 ■ its own class.
 */
static BubasLanguage.Builder dated() {
    return telling()
            .defineOpaqueTypeVia("Date", DateDoc.class)
            .defineFunction("SUBMITTED_ON", SubmittedOn.class)
            .defineFunction("DAYS_BETWEEN", DaysBetween.class);
}
```

The interface exists only to hold the words. It costs one file, and it is the escape route rather than the road.

Both forms produce the same vocabulary entry. A reader of chapter 11's document cannot tell which was used, and should not be able to.

When not to make something opaque

Not every value from your domain needs to be a type.

If a thing is genuinely a number, a piece of text, or a yes-or-no, expose it as one. A claim's reference number is a `STRING`; making it an opaque `ClaimReference` buys nothing and costs every rule the ability to put it in a message.

The test is whether rules need to *do* anything with the value beyond passing it along. If they only ever hand it to other operations, opaque is right. If they need to compare it, print it, or build a message out of it, it is a value and should be one.

The middle case — a thing rules mostly pass along but occasionally need to name — is handled by an operation, not by making the type transparent. `MERCHANT_OF(line)` returns a `STRING` precisely so that `Item` need not be readable.

What is coming

Types are the nouns. The next two chapters are the verbs: functions that answer questions, and the pattern language that turns a class into a statement.

CHAPTER 23

Defining functions

Operations that answer questions: handler classes, the context they are given, variable arity, and wildcard parameters. Which work belongs behind a function, and the signal that you are about to put business logic somewhere your experts cannot see it.

The shape

A function is a class with a `call` method:

```
@BubasDescription("What a claim comes to in total, in euro.")
public static final class TotalOf {
    public BigDecimal call(Context ctx, Report claim) {
        return claim.total();
    }
}
```

Registered with `.defineFunction("TOTAL_OF", TotalOf.class)`, and that is the whole contract. No interface, no base class, no annotations beyond the description.

BUBAS reads the signature to work out the vocabulary entry. `Context` is always first and is not a BUBAS parameter. The rest map by their Java types: `long` is `INTEGER`, `BigDecimal` is `DECIMAL`, `String` is `STRING`, `boolean` is `BOOLEAN`, and a registered class is its opaque type. The return type becomes what the function answers, and `void` makes it a statement-form call — chapter 4's `NOTE`.

Parameter names matter. They appear in the vocabulary document and in every type error: `AMOUNT_OF takes Item for 'line', but was given Report`. Name them for the domain, not `arg0`.

What the context gives you

`Context` is how a handler reaches the world outside itself: `service(Class)` for whatever the application registered, `log`, `debug`, `error`, and `mathContext()` for the division rules of chapter 3.

It comes in two pieces. `CoreContext` has everything but the services — the rounding policy, the log, and `error` — and `Context` adds `service`. Take `Context` unless you have a reason not to; the next section is the reason.

What it does not give you is access to the program. A handler cannot read the rule's variables, cannot know which line called it, and cannot change what happens next. It answers the question it was asked. That restraint is what makes chapter 21's threading story simple.

Functions the compiler may call

A function is a black box to the compiler. It might read a database, a clock or a feature flag, and nothing about `long call(Context, String)` says which — so a call is never worked out early, however arithmetical it looks.

A function that genuinely answers from its arguments alone can say so, and the standard module's `TO_INTEGER` does:

```
@BubasMemoizable
public final class ToInteger {

    public static final String NAME = "TO_INTEGER";

    public long call(CoreContext ctx, String s) {
        try {
            return Long.parseLong(s.trim());
        } catch (NumberFormatException e) {
            ctx.error("'" + s + "' is not an INTEGER");
            throw new IllegalStateException("unreachable: error()
            ■ throws", e);
        }
    }
```

The compiler may then call it while compiling, when every argument is a value it already knows. `TO_INTEGER("5")` becomes 5, and — since chapter 8's refusals follow values — a test on the result is a test with an answer, and refused.

Three things are worth knowing before you reach for it.

Half of it is not a promise at all. Look at the first parameter: `CoreContext`, not `Context`. It carries the rounding policy, the log and `error`, and it has no `service` method on it — so a memoizable function that tries to reach the application does not compile, in Java, in your IDE, anything else happens.

Writing `Context` there is refused when the language is sealed, at startup, rather than on whichever compilation first happens to know all the arguments.

The other half is a promise, and nothing checks it. A clock read through a static field, a file, a cached lookup: those the compiler cannot see. A function that declares this falsely will answer at compile time with a value a run would not have produced, and nothing will say so. When in doubt leave it off; all you lose is a fold.

It only ever applies to plain values. Every parameter and the result must be `INTEGER`, `DECIMAL`, `STRING` or `BOOLEAN`. An array is a store, an opaque value is a Java object, and neither is something a compiled program can hold, so a function touching one is never called early however pure it is. `TOTAL_OF(claim)` takes a `Report` and would not qualify — which is just as well, since it reads one.

Refusing is allowed, and becomes a compile error. `ctx.error` during a compile-time call fails the compilation at the line of the call. That follows from the promise rather than contradicting it: a function that answers the same way every time and refuses these arguments now would refuse them on every run, so the compiler is giving the same answer earlier.

Logging is allowed too, and the line is thrown away. It decides nothing, so it is no reason to decline the fold — but the run that would have written it may happen a thousand times or never, and compiling is not one of them. Do not put anything in such a function's log that you expect to count.

Any number of arguments

```
/**
 * Variadic: BUBAS sees {@code NOTIFY(people STRING...)} and calls it
 ■ with a spread list, never
 * with an array. An embedder who wants an array declares an array
 ■ parameter instead.
 */
@BubasDescription("Tells one or more people about a claim. Answers
■ nothing.")
public static final class Notify {
    public void call(Context ctx, String... people) {
        ctx.log("NOTIFY", "told " + String.join(", ", people));
    }
}
```

A Java varargs parameter becomes a variadic operation. A rule calls it with as many as it likes — `NOTIFY "the claimant"` on one path, and `NOTIFY approver, "the claimant", "the finance inbox"` on another, both in the program above.

Spread only. `NOTIFY(people)` where `people` is an array does not compile, even though Java would accept it, because allowing both would revive an ambiguity nobody wants. An embedder who genuinely wants an array declares an array parameter instead.

Every argument is type-checked against the element type, so the error names the position that was wrong rather than reporting a count.

Arguments of any type

```
/**
 * A wildcard parameter: BUBAS sees {@code RECORD(label STRING, value
 ■ ANY)} and accepts any
 * type in the second position. The handler asks the value what it is
 ■ rather than being told by
 * its own signature.
 */
@BubasDescription("Writes a labelled value into the decision record,
■ whatever kind of value it is.")
public static final class Record {
    public void call(Context ctx, String label, Value value) {
        ctx.log("RECORD", label + " = " + value.as(Object.class) + "
        ■ (" + value.type() + ")");
    }
}
```

Declaring a parameter as `Value` makes it `ANY`: it accepts every type, and the handler asks the value what it is rather than being told by its own signature.

```
TellPeople(claim = report 1 (Alice), limit = 200.00)
    total = 128.40 (DECIMAL)
    over the limit = false (BOOLEAN)
    told the claimant
    approved report 1 (Alice) for 128.40
    => TRUE

TellPeople(claim = report 2 (Erin), limit = 200.00)
    total = 450.00 (DECIMAL)
    over the limit = true (BOOLEAN)
    budget left = 2500.00 (DECIMAL)
    told the line manager, the claimant, the finance inbox
    escalated report 2 (Erin) — over the 200.00 limit
    => FALSE
```

A decimal and a boolean went into the same operation, and each rendered itself.

`ANY` is for **parameters only, never returns**. An operation that answers `ANY` would push the type question into the rule, where there is nothing to do about it — a rule cannot ask a value what it is. Answer a real type, or answer text.

Use it sparingly. Logging, recording and rendering are the honest cases: operations whose whole job is to accept whatever they are given. An operation that behaves *differently* depending on the type it receives is a design that wants to be several operations.

What belongs behind a function

This is the judgement that decides how good the language feels to use.

Anything the domain has a name for. If people say "what did this claim come to", there should be an operation called that. The test is whether the phrase exists before you write the code.

Anything computational. Route optimisation, currency conversion, a scoring model, a date calculation. Chapter 9 made the argument from the rule-writer's side: wanting a data structure means you have left business logic. This is the other half of it — that work belongs here, where it can be tested properly in Java.

Anything that would otherwise be repeated. If three rules compute the same thing from the same two operations, that computation is a piece of domain knowledge living in the wrong place. Promote it.

The signal you are hiding the rule

The opposite mistake matters more, and it is easy to make while feeling productive.

An operation named `CHECK_EXPENSE_POLICY(claim) -> BOOLEAN` would work. It would also move the entire policy into Java, where the person who owns it cannot read it, and leave the BUBAS rule as a single line that calls it. Everything this book argues for would have been given up in one registration.

The test is: **does this operation encode a decision somebody could disagree with?**

`TOTAL_OF` does not — adding up lines is arithmetic. `ANOMALY_SCORE_OF` returns a score and leaves the decision in the rule, which is chapter 25's whole subject. `CHECK_EXPENSE_POLICY` is a decision wearing a function's clothes.

When you find yourself about to expose one, the question to take back to whoever asked is usually "which part of this is the policy?" — and the answer is what stays in the rule.

What is coming

Functions answer. The next chapter is the pattern language: how a class becomes a statement, with keywords in the middle and variables it can write into.

CHAPTER 24

Defining commands

The pattern language: how `REJECT claim, "over the limit"` *becomes a statement, with placeholders, kinds, type constraints, and conditions on what a variable must be before and after. How the analysis at sealing time proves no two commands can ever match the same line.*

A statement is a shape

A function is identified by its name and called with brackets. A statement is matched as a **whole line**, against a shape you write:

```
@BubasDescription("Approves a claim for payment.")
public static final class Approve {
    public void call(StatementContext ctx, ExpressionArg claim) {
        final var filed = claim.evaluate().as(Report.class);
        ctx.log("DECISION", "approved " + filed + " for " +
        ■ filed.total());
    }
}
```

```
.defineStatement("APPROVE {expression/Report:claim}", Approve.class)
```

That second line is a call on the builder, alongside the `defineFunction` and `defineOpaqueType` calls of chapter 21 — a statement is registered exactly like anything else, and the only difference is that its name is a shape rather than a word.

Everything outside braces is a keyword the rule must write literally. Everything inside is a **placeholder**: a hole, with a kind, usually a type constraint, and a name.

That is why statements can have words in the middle. `ROUTE claim TO approver AT centre` is one statement whose shape happens to include `TO` and `AT`, and the words carry meaning for the reader rather than being punctuation.

Placeholders

The full form has four zones, all optional:

```
{ prefixes > kind[/constraint] : name > postfixes }
```

Kind is what the hole accepts. Five of them, and the two that matter most are `expression` — anything that produces a value, handed to your handler unevaluated — and `identifier`, a bare variable name. `literal` insists on a compile-time constant, `type` takes a type designator, and `var` is `identifier` with an appetite for an index, so `totals[3]` matches it.

Constraint is the type: `/Report` means the expression must be a `Report`, `/STRING` a string. It can also name another placeholder in the same pattern, meaning *the same type as that one* — which is how `DECLARE` ties a variable's type to the type designator later in its own line.

Name is what the handler's parameter is called and what the vocabulary document shows. A placeholder may not be named after a type, checked at `seal()`, for the same reason chapter 22's variables cannot be: `/Report` would otherwise have two readings.

Prefixes and postfixes are conditions on a variable, and they are the interesting part.

Before and after

```
/** Stage 6: one operation, two answers, and the budget they point
■ at. */
static BubasLanguage.Builder routing() {
    return screening()
            .defineOpaqueType("CostCentre", CostCentre.class)
            .defineFunction("BUDGET_LEFT", BudgetLeft.class)
            .defineStatement("ROUTE {expression/Report:claim}"
                    + " TO {new > identifier/STRING:approver >
                    ■ initialized}"
                    + " AT {new > identifier/CostCentre:centre >
                    ■ initialized}", Route.class);
}
```

`{new > identifier/STRING:approver > initialized}` says three things.

`new >` is a **precondition**: this variable must not exist yet. The statement creates it.

`identifier/STRING:approver` is the hole itself.

`> initialized` is a **postcondition**: after this statement runs, the variable holds a value. That is what lets chapter 8's flow analysis know the rule may read `approver` on the next line, without knowing anything about what your handler does.

This is how `DECLARE` works too, and it is why chapter 4 could say that declaration is not syntax. `DECLARE {new > identifier/T:name > declared} {type:T}` is an ordinary pattern shipped in a module you may decline to install.

One consequence to know before it bites: **a statement that declares may only appear at the top level of a program.** Not inside an `IF`, not inside a loop. BUBAS has no local variables, so a declaration inside a block would look scoped while being global, and the message says so.

Writing into a variable

A handler receives a `VariableArg` for a placeholder it writes, and `set` puts a value in:

```
/**
 * Two answers from one lookup, which is the only reason this is a
 ■ command rather than a
 * function. Who signs and which budget it lands on are decided
 ■ together, by one reading of the
 * approval policy; asking twice could get answers from two different
 ■ readings.
 */
@BubasDescription("""
        Works out who has to approve a claim and which budget it will
        ■ be charged to.
        Both come from one reading of the approval policy, so they
        ■ are always consistent.
        """)
public static final class Route {
    public void call(StatementContext ctx, ExpressionArg claim,
    ■ VariableArg approver,
                     VariableArg centre) {
        final var filed = claim.evaluate().as(Report.class);
        approver.set(filed.total().compareTo(new BigDecimal("1000"))
        ■ > 0
                ? "the finance director" : "the line manager");
        centre.set(new CostCentre("CC-" + filed.id, new
        ■ BigDecimal("2500.00")));
    }
}
```

`Route` there is a static nested class, and so is every handler in this book. Nothing requires it — a handler may be a top-level class in a file of its own — but a vocabulary of a dozen operations is a dozen small classes, and keeping them nested inside the class that registers them puts the whole language in one file where it can be read as one thing. The only requirement is that a nested handler be **static**: the runtime constructs it with no arguments, so it cannot be an inner class needing an enclosing instance.

An `ExpressionArg` is different: it arrives **unevaluated**, and calling `evaluate()` is what runs it. That is deliberate, and it is the one power a statement has that a function does not — a statement decides *whether* and *when* its arguments run. `APPROVE` evaluates its claim once. A conditional statement could evaluate one branch and not the other.

Use it rarely, and only for a reason you can name. An operation whose arguments may or may not run is harder to reason about than one that always evaluates, and chapter 12's reader has to work out which it is.

`ROUTE` is a case where the choice makes itself. It cannot decide anything without knowing which claim it is routing, so it evaluates its expression immediately and unconditionally — and if it did not, it would have nothing to write into either of its two targets. The power exists for the statement that genuinely needs to skip an argument; a statement that needs all of its arguments should just take them.

Saying what a command puts there

`ROUTE` writes two variables, and the compiler knows only that. It cannot know *what* — the approver comes out of whatever `Route` does with the claim, and there is nothing in the pattern, or in Java, that could tell it.

That is the ordinary case and needs nothing. It is worth knowing about the exception, because chapter 8's refusals run on it.

A command that does not compute but *copies* can say so. Assignment is such a command — it is an ordinary statement in the standard module, not a language built-in — and it declares what lands where:

```
@BubasAssigns(target = "name", value = "value")
public final class Assign {

    public static final String PATTERN =
            "{mutable:declared > var:name > initialized} =
            ■ {expression/name:value}";
```

`target` names the variable placeholder, `value` the expression placeholder whose value it receives. That one line is why the compiler knows that after `strict = TRUE` the variable holds `TRUE`, and therefore why the `IF` four lines later has an answer.

Three properties of the declaration are worth stating plainly.

It is optional and it is a claim. A command that declares nothing is not deficient; it is opaque, which is what nearly every command should be. A command that declares this and then writes something else has lied to an analysis, and nothing checks it.

It repeats. One statement may fill several variables, and each occurrence names one target. A command filling two and declaring one is describing itself accurately — the declared variable is known afterwards, the other is not. No two occurrences may name the same target; `seal()` refuses that, because two claims about one variable cannot both be the story.

Anything you hand a command is assumed written. Nothing at run time stops a handler calling `set` on a variable its pattern only claimed to read, so the compiler forgets the value of every variable a statement touches, and puts back only what was declared. If your command is handed something it merely reads, the compiler is being pessimistic about it, and that costs nothing but a refusal it will not make.

Why this command is a command

Worth restating, because the temptation runs the other way.

`ROUTE` produces two values, decided together by one reading of the approval policy. Had it produced only the approver, it should have been `approver = FIND_APPROVER(claim)` — a writing command with a single target is a function wearing a disguise, harder to read, harder to compose, and it declares a variable as a side effect of being called.

More than one answer, decided together. That is the test — not "it writes rather than returns". Writing into a variable is only interesting when there is more than one place to write to; with one, returning the value says the same thing and reads better.

Sealing proves the shapes cannot collide

Two statements whose shapes could both match one line would make a program's meaning depend on which was registered first. BUBAS refuses that outright, at `seal()`.

The analysis is exhaustive rather than heuristic: it treats each pattern as an automaton and looks for any line both would accept. If one exists, sealing fails with a message naming both patterns, at startup, before any program has been written.

Two practical consequences. **You cannot ship an ambiguous vocabulary** — not to a test environment, not to production. And **adding a statement can break sealing**, which is the right time to find out: a new pattern that overlaps an existing one is caught by the build that added it.

If it fires, the fix is nearly always to add a distinguishing keyword. `SEND claim TO "finance"` and `SEND claim, "finance"` collide; `SEND claim TO "finance"` and `NOTIFY claim, "finance"` do not.

Designing shapes people can read

Three habits, in decreasing order of how much they matter.

Read it aloud. `ROUTE claim TO approver AT centre` is a sentence. `ROUTE claim, approver, centre` is an argument list. Both work; only one tells the reader what the second and third things are for.

Put the keywords where the meaning changes. `TO` and `AT` are doing real work — they say which hole is the person and which is the budget. A keyword that carries no such information is noise.

Keep the set small. The overlap analysis is exhaustive, so a large command set stays correct, but chapter 11's reader has to hold it in view as one thing. Ten or a few tens is the size this is built for.

CHAPTER 25

Exposing a model

Putting an LLM behind an operation, and the design rule that decides whether it was worth doing: return a score, never a verdict. Advisory outputs leave the decision with the expert; verdict outputs move it to the model and reduce your language to glue. And the limit worth stating plainly: this is the one operation whose behaviour can change without anybody editing a line of code.

An operation like any other

Putting an LLM behind an operation changes nothing about how the language sees it. It is a function; it takes an argument; it answers a value. Everything that is different about it — that it costs money, that it takes a second, that it will not say the same thing twice — is behind the boundary, where the rule cannot see it and does not need to.

What that leaves you is one decision, made once, when you write the signature. It is the subject of this chapter, and it decides whether the arrangement was worth building.

Here is the operation this book uses:

```
/**
 * Stands in for a model. A real implementation would send the line
 ■ to a service and return
 * what it answered; this one is a fixed rule of thumb, because the
 ■ build must run offline for
 * anyone, forever, and a book whose examples change between
 ■ printings is no use.
 * <p>
 * It returns a <em>score</em>, never a verdict. What counts as too
 ■ high is a threshold in the
 * rule, where the person accountable for the policy can read and
 ■ change it. See
 * {@code DOCUMENTATION/AUTHORING.md} D10.
 */
@BubasDescription("""
        How unusual a line of spending looks, from 1 (ordinary) to 10
        ■ (very unusual).
        It is an opinion, not a decision: the rule decides what score
        ■ is too high.
        """)
public static final class AnomalyScoreOf {
    private static final BigDecimal LARGE = new BigDecimal("100");
    private static final BigDecimal VERY_LARGE = new
    ■ BigDecimal("500");
    private static final BigDecimal LAVISH_MEAL = new
    ■ BigDecimal("75");

    public long call(Context ctx, Item line) {
        var score = 1L;
        if (line.amount().compareTo(LARGE) > 0) {
            score += 2;
        }
        if (line.amount().compareTo(VERY_LARGE) > 0) {
            score += 3;
        }
        if (!line.hasReceipt()) {
            score += 2;
        }
        if ("meals".equals(line.category()) &&
        ■ line.amount().compareTo(LAVISH_MEAL) > 0) {
            score += 3;
        }
        return Math.min(score, 10);
    }
}
```

That implementation is a fixed rule of thumb, because a book's examples must run offline and give the same answer in every printing. A real one would send the line to an AI service and return what came back. The shape is what matters, and the shape is the subject of this chapter.

Registering it is unremarkable — `.defineFunction("ANOMALY_SCORE_OF", AnomalyScoreOf.class)` — and that unremarkableness is the point. From the language's side there is nothing special about an operation backed by a model. Everything that is different about it lives behind the boundary.

The rule that decides whether it was worth doing

It returns an `INTEGER` from 1 to 10. It could have returned a `BOOLEAN`.

`SHOULD_ESCALATE(claim) -> BOOLEAN` would work, would be less code in every rule, and would be a serious mistake.

With a score, the policy stays in the rule. `IF worst >= flagAt THEN` is a line a finance manager reads, disagrees with, and changes. When an auditor asks why this claim was flagged and that one was not, the answer is a threshold somebody chose, in an artefact they can be shown.

With a verdict, the policy moves behind the operation. Not into the dark — it is in the Java, under version control like everything else, and an engineer can read it, review it and explain it. The difference is who. The person who owns expense policy cannot read Java, and they are the one the auditor will ask and the one accountable for the answer. Every question about the rule now needs an engineer to translate it, and the BUBAS program that existed so that expert could own the rule has been reduced to glue between a service and a database.

Stated generally, and worth keeping when somebody asks you to add an operation:

> ***Advisory outputs — scores, classifications, extractions — leave the decision with the expert. Verdict outputs move it to the model.***

This is not an argument against models. It is an argument about which side of the boundary the decision sits on.

Recognising a verdict in disguise

A boolean return is the obvious case. Two subtler ones are worth watching for.

A score with only two values. An operation returning 1 or 10 and nothing between is a boolean that has learned to count. If the model genuinely only distinguishes two states, say so and accept that the decision has moved.

An operation named for the decision. `RISK_OF(claim) -> STRING` returning `"HIGH"` or `"LOW"` is a classification, which is advisory — but if the rule's only possible response is to escalate anything `"HIGH"`, the threshold has been chosen by whoever wrote the classifier. Ask what the rule would do with a third value. If there is no sensible answer, the categories are the policy.

The test that cuts through both: **could a person move the line without retraining anything?** With a score and a threshold, yes. That is the property you are buying.

Practical consequences to design for

Calls cost time and money. A rule may call the operation once per line, and chapter 10 pointed out that the obvious way to write the loop calls it twice per line. You cannot fix that from here — the rule is not yours — but you can make it cheap to get right by keeping the operation's cost visible in its description, and you can cache inside the handler where the domain permits.

Failure needs a decision, and it is yours. A service can be slow, down, or answer nonsense. The rule has no vocabulary for any of that. So the handler decides: throw, and the run fails with a diagnostic; or return a conservative value, and the rule proceeds on an assumption nobody wrote down. Neither is obviously right, and choosing silently is what makes it wrong. Chapter 30 is about who sees the consequences.

Determinism is gone, and it propagates. Every rule calling this operation becomes untestable by execution, which is chapter 19's subject and a real cost to the people writing rules. It is worth paying once, for an operation that earns it, and not twice.

The honest part

The vocabulary bounds what a program can **name**. It does not bound what a named thing **does**.

That has been true of every operation in this book, and it is worth repeating here because this is where the gap is the widest. An operation described as returning an opinion about a line of spending could, behind that description, send the whole claim to a third party, retain it, log the claimant's name somewhere it should not be, or cost a euro a call. The description is what somebody wrote; the behaviour is what somebody built.

Worse, it is the one operation whose behaviour can change **without anybody editing a line of code**. A model is replaced, a prompt is tuned, a provider changes a default — and every rule depending on it now decides differently. The review checksum of the next chapter will not notice, because the vocabulary's shape has not changed. Nothing in BUBAS will notice.

So the discipline has to come from outside the language:

- **Pin the model version** in the handler, and treat changing it as a code change with a review.
- **Record what was asked and what came back**, alongside the decision, so an outcome can be explained six months later.
- **Test the boundaries**, as chapter 19 describes, so that a shift in behaviour has some chance of showing up as a failing case rather than as drift nobody sees.

None of that is BUBAS's job. All of it is yours, and it is the price of the one operation in the vocabulary that has an opinion.

What is coming

Every operation in this book carries a description written for somebody who will never see the Java. The next chapter is about writing those, exporting them, and knowing when a vocabulary has changed shape since anybody last looked at it.

CHAPTER 26

Describing and exporting

Writing the vocabulary document that chapter 11 taught people to read: descriptions, where they live so that domain classes stay clean, the export itself, and a checksum that tells you when a reviewed vocabulary has changed shape since anybody looked at it.

The document is generated, so it cannot lie about what exists

Chapter 11 showed a reader how to find out what a language can do. This is where that document comes from:

```
final var export = VocabularyExport.of(language);
final var document = export.asMarkdown();
```

It is built from the sealed language, so it cannot describe an operation that does not exist and cannot omit one that does. There is no wiki to keep up to date and no comment block to drift.

`asJson()` gives the same content in a form a tool can read, which is what you want when the consumer is an LLM being asked to write rules rather than a person reading them — chapter 32's subject.

It refuses to be built out of nothing

`VocabularyExport.of` throws if anything is undescribed, listing everything that is:

```
nothing describes:
        opaque type Ledger
    An export says what a vocabulary means, so it cannot be built out
    ■ of things nobody has said anything about. Add @BubasDescription
    ■ to each.
```

That is a deliberate refusal rather than a nicety. A document with holes in it is worse than no document, because a reader who finds three operations undescribed stops trusting the other twenty.

The practical consequence: **write a test that builds the export**. A vocabulary that cannot describe itself then fails, and the moment it fails is the commit that added the operation rather than the afternoon somebody first wanted the document.

Where descriptions live

```
/** A value BUBAS holds and passes but cannot look inside. */
@BubasDescription("""
        One employee's expense claim for a trip or a period.
        A program is given one to decide about; ask TOTAL_OF what it
        ■ comes to.
        """)
public static final class Report {
    final long id;
    private final String employee;
    private final LocalDate submitted;
    private final List<Item> items;

    Report(long id, String employee, LocalDate submitted, List<Item>
    ■ items) {
        this.id = id;
        this.employee = employee;
        this.submitted = submitted;
        this.items = items;
    }

    BigDecimal total() {
        return items.stream().map(Item::amount).reduce(BigDecimal.ZERO
        ■ , BigDecimal::add);
    }

    @Override
    public String toString() {
        return "report " + id + " (" + employee + ")";
    }
}
```

For a **function or statement**, the annotation goes on the handler class, which you own and which exists only for this purpose.

For a **type**, it goes on the class itself, as above, and `defineOpaqueType` takes it from there.

The exception is a type whose class cannot carry the annotation — one from a library, or a domain model shared with consumers that must not depend on BUBAS. Describe those on an empty interface carrying

`@BubasDescribes(TheClass.class)` and register with `defineOpaqueTypeVia`. Chapter 22 has the argument for when that is worth doing; the export cannot tell the difference either way.

Writing them

The audience is somebody who will never see the Java, and that changes what a good description says.

Say what it is in the business, not what it holds. Look again at `Report` above: one employee's claim for a trip or a period, and which operation to ask about it. Not a list of fields — a reader cannot reach them anyway.

Point at the neighbouring operations. `Item`'s description names `AMOUNT_OF`, `CATEGORY_OF`, `MERCHANT_OF` and `HAS_RECEIPT`, so a reader who has just met a line knows what to ask it. In a document read by someone assembling a rule, those signposts do more than prose does.

Say what is decided elsewhere. The anomaly score's description ends *"It is an opinion, not a decision: the rule decides what score is too high."* That sentence is doing chapter 25's work in the place where a rule-writer will actually meet it.

Do not describe the implementation. Whether a total is cached, which service is called, how a score is computed — none of it can be acted on by the reader, and all of it will be wrong eventually.

Write in the second person or not at all. These are instructions to somebody using the vocabulary, not notes to yourself.

Knowing when a vocabulary has moved

A description is prose, and prose goes stale silently. There is a mechanism for the part of it that can be checked mechanically.

`Surface.checksum(Class)` hashes a class's public surface — its methods and their shapes. Recording it with `@BubasReviewed` says *somebody read this and agreed with what it said, at this shape.*

There are three states, and the difference between them is the whole design.

No annotation. Nothing is checked. Reviewing is opt-in per class, so a project that has not adopted it pays nothing.

An empty value, which is how you start:

```
/** A type whose description has never been reviewed. The empty value
■ asks for a checksum. */
@javax0.bubas.api.BubasDescription("A cost centre a claim is charged
■ to.")
@javax0.bubas.api.BubasReviewed("")
public static final class Budget {
    public String code() {
        return "CC-1";
    }
}
```

The export refuses, and tells you what to write:

```
write E69EB45F13B8D06C into @BubasReviewed on
■ javax0.bubas.doc.expense.VocabularyTest$Budget
```

Note that it *reports* the checksum rather than writing it into your source. A build that edits its own files to make itself pass has stopped being a check.

A value that no longer matches. Once the class changes shape, the export says so, shows you the surface as it now stands, and names what to do:

```
javax0.bubas.doc.expense.VocabularyTest$Stale has changed since its
■ description was reviewed. Its public surface is now:
        java.lang.String code()
    Re-read the description, then write E69EB45F13B8D06C into
    ■ @BubasReviewed on javax0.bubas.doc.expense.VocabularyTest$Stale
```

Pasting the new checksum in is a two-second edit, and that is exactly the risk: the mechanism only works if somebody re-reads the description first. It buys you the prompt, not the review.

What it catches: an operation gaining a parameter, changing its return type, or acquiring a new public method. Real changes that plausibly invalidate a sentence somebody wrote.

What it cannot catch, and this is the important half:

- **A behaviour change behind an unchanged signature.** The commonest way for a description to become false, and the checksum is blind to it by construction.
- **A model swapped out behind the same operation.** Chapter 25's warning, restated.
- **A description that was never accurate.**

So the checksum is a prompt for a human review, not a substitute for one. Treated as the latter it is worse than nothing, because it produces the feeling of having a process.

An annotation that is absent is ignored rather than treated as a failure. That is deliberate: a vocabulary should be usable before anybody has reviewed it, and adopting the review discipline should be a choice you make when you are ready to keep it.

Who should read it, and when

The export is not documentation you write and file. It is the artefact chapter 1's finance manager reviews **before any rules exist**.

That is the moment it pays for itself. A vocabulary reviewed at the start is a set of operations somebody with domain authority agreed the domain has. Reviewed after fifty rules depend on it, the review can only ratify.

Three moments worth building a habit around: when the vocabulary is first designed, when an operation is added, and when the checksum says a shape moved. The first is the one people skip and the one that matters most.

What is coming

The vocabulary is defined, described and reviewable. The next chapter is the last piece of building it: teaching BUNIT about statements of your own, so that the tests in Part 2 work for operations this book never saw.

CHAPTER 27

Extending BUNIT

Mock commands of your own, for operations that write values rather than return them. How interception works, what the consistency checker needs to be told about a command it has never seen, and why that is expressed as annotations rather than an interface.

Most of the time you need none of this

BUNIT ships a complete set of statements — `IS MOCKED`, `RETURNS`, `SETS`, `ARGUMENT`, `RUN`, the expectations — and they work against any vocabulary. Nothing in Part 2 was specific to expense approval.

This chapter is for when you want a testing statement of your own: something that reads better for your domain, or arranges a world your operations need and the standard statements express clumsily. It is a small chapter because the mechanism is small.

How mocking works at all

Worth understanding before extending it, because it explains why extension is possible.

BUNIT does not compile a parallel mocked language. It compiles the **real** subject, against the **real** vocabulary, and then puts an interceptor between the running program and the handlers. When the subject calls `TOTAL_OF`, the interceptor decides whether to answer from the test or let the real handler run.

That choice matters more than it looks. A parallel mocked language would be a second implementation of your vocabulary, and the two would diverge — a rule would pass its tests and fail in production because the mocked `Report` behaved differently from the real one. Interception makes divergence impossible: there is one language, and tests differ only in who answers.

It also means a BUNIT test is a real BUBAS program, which is why chapter 18's checker can trace its paths, and why a test can contain an `IF` or a loop.

A statement of your own

A BUNIT statement is an ordinary BUBAS statement — chapter 24's pattern language, unchanged — plus annotations telling the framework what role it plays. Here is the standard `ARGUMENT`, which is as simple as they get:

```
@NamesParameter("name")
public final class Argument {

    public static final String PATTERN = "ARGUMENT
    ■ {literal/STRING:name} IS {expression:value}";

    public void call(StatementContext ctx, String name, ExpressionArg
    ■ value) {
        Mock.recorder(ctx).argument(name, value.evaluate());
    }
}
```

The pattern is a pattern. The handler talks to the recorder. The annotation says which placeholder holds the name of a subject parameter.

What the annotations tell the checker

The consistency checker of chapter 18 has to reason about a statement it has never seen. It cannot read your handler, so you tell it what the statement does:

Annotation	Says
`@NamesTarget`	which placeholder holds the operation this statement is about
`@DeclaresMock`	this statement puts that target into the mocked set
`@SuppliesVariable`	this statement supplies a value the mocked command would have written
`@SuppliesResult`	this placeholder holds what a mocked function answers
`@MatchesArguments`	these placeholders are the arguments of the call being described
`@CountsArguments`	this placeholder holds a call whose own arguments are the arguments

Annotation	Says
`@NamesParameter`	this placeholder names a subject parameter being supplied
`@Act`	this is the statement that runs the subject
`@Expectation`	this examines what happened, so it may only appear after the act

With those, the checker can decide whether a test is meaningful without knowing anything about your domain. `@Expectation` alone is what makes "you cannot ask before you run" enforceable for a statement nobody at BUNIT wrote.

Why annotations and not an interface

The obvious design is an interface — `MockVariableSetter` with a `variableName()` method — and it is the wrong one.

What the checker needs is **static** information: which placeholder plays which role, known before anything runs. An interface is implemented by instance methods, which means calling a handler to ask what it would do. Annotations are read from the class, at seal time, without constructing anything.

There is a second reason that shows up in practice. A single statement can supply more than one variable — chapter 24's `ROUTE` writes two — and `@SuppliesVariable` is repeatable, so that says naturally what an interface would have needed a collection to express.

When to write one, and when not

Write one when your domain has an arrangement the standard statements make awkward and your tests keep repeating. A statement that sets up a plausible claim in one line, where four `RETURNS` would be needed, will pay for itself across a hundred tests.

Do not write one to make tests shorter in general. Every custom statement is vocabulary that a new reader must learn before they can read a test, and chapter 12's whole argument is that tests are read by people who did not write them. The standard set is small enough to learn once and appears in every BUBAS project; yours appears in one.

The bar is roughly: would somebody unfamiliar with your codebase understand a test using it, on sight, without asking? If not, four ordinary lines are better than one clever one.

What is coming

That is the vocabulary complete: types, functions, statements, descriptions, and the testing statements to go with them. The rest of Part 3 is about running it — wiring, threading, failure, where rules come from, and what changes when a model writes them.

CHAPTER 28

Wiring

Services, logging, configuration, and the context handlers are given. Keeping handlers thin, and where the application's real work should sit so that the vocabulary stays a vocabulary.

Handlers must not own anything

A handler class is constructed by BUBAS, not by you. It has no constructor you control, no fields you set, and no way to be injected into.

That is not an oversight, and working around it is the first mistake people make. A handler that held a database connection would be a handler with a lifecycle, and the whole point of chapter 21's three objects is that only one of them has a lifecycle worth thinking about.

So a handler reaches out instead:

```
// snippet: thin-handler
/** A handler translates and delegates. It owns nothing and has no
■ lifecycle. */
public static final class TotalOf {
    public BigDecimal call(Context ctx, Expense.Report claim) {
        return ctx.service(ClaimStore.class).totalOf(claim);
    }
}
```

`Context.service(Class)` returns whatever the application registered on the interpreter for this run. There is a qualified form, `service(Class, String)`, for when one type has several instances — two databases, two clients.

Registering them

Services go on the interpreter, which means **per run**:

```
// snippet: per-run-wiring
/** Services, arguments and the logger are supplied per run, never
■ per language. */
static Value decide(BubasProgram program, ClaimStore store,
■ Expense.Report claim,
                    BigDecimal limit,
                    ■ java.util.function.BiConsumer<String, String>
                    ■ auditLog) {
    return Interpreter.of(program)
            .registerService(ClaimStore.class, store)
            .argument("claim", claim)
            .argument("limit", limit)
            .logger(auditLog)
            .run();
}
```

Per run rather than per language, and that is deliberate. It means a request-scoped transaction, a tenant-specific store or a test double can be supplied for one execution without touching anything shared. The language and the program stay immutable; everything that varies varies here.

The cost is that a service forgotten is discovered when a handler asks for it. Registering the same set in one place — a small factory that turns a claim into a configured interpreter — is worth doing on the first day rather than the fortieth.

What a run is allowed to spend

Two things an application can cap, and both belong on the interpreter because both vary by caller — a nightly batch and a request handler do not deserve the same budget:

```
Interpreter.of(program)
        .maxSteps(1_000_000)
        .maxArrayLength(100_000)
```

`maxSteps` counts statements executed and loop passes taken. Chapter 8's compiler already refuses the loops it can *prove* never end, but a loop that stops when a service says so is not one of those, and the service can be wrong. `maxArrayLength` caps what a single `DECLARE items[n] Order` may bring into existence, which matters because `n` is an expression and a wrong figure from upstream is an allocation nobody intended.

Both are unlimited unless you say otherwise, and a run inside its budget cannot tell they are there. Set them anyway. A rule is written by somebody who is not thinking about budgets, which is the arrangement this whole book argues for, and it only works if somebody else is.

One obligation comes with the second. **A command that allocates has to ask** — the array limit is readable through `CoreContext.maxArrayLength()`, and the standard `DECLARE` consults it before taking the memory. The runtime cannot do this for you: an array that has arrived in a variable is already allocated, so a check there would catch the mistake after paying for it. If chapter 24's vocabulary grows a statement that makes an array, it asks first.

Rounding, which is not per run

One setting deliberately does not go on the interpreter:

```
// snippet: rounding
/**
 * The rounding policy belongs to the language, not to the run.
 ■ Sixteen significant digits and
 * banker's rounding, decided once, at startup, for every rule this
 ■ language ever compiles.
 */
static BubasLanguage roundedToTheCent() {
    return Expense.approving()
            .mathContext(new MathContext(16, RoundingMode.HALF_EVEN))
            .seal();
}
```

Chapter 3 said the application draws the line on division and every rule follows suit. This is where it draws it: on the builder, sealed with everything else, before a single rule is compiled.

It could have been per run, and was once. The reason it is not is worth a paragraph, because it is the same reason this chapter keeps insisting on immutability. A rounding policy on the interpreter means the same rule can settle money one way in your test and another way in production, with nothing anywhere comparing the two — and the BUNIT suite of chapter 12 onwards takes the language, so it would have inherited the wrong one silently. Rounding is an accounting decision, one rule set answers to one, and it belongs where the vocabulary is decided rather than where a request is served.

The practical consequence is that a rule needing different rounding for one calculation asks for it explicitly, with an operation that says so. That reads better in a rule than an invisible global, which is the trade this book keeps making.

Logging

`logger(BiConsumer<String, String>)` receives every `ctx.log(level, message)` a handler makes.

Two things worth deciding early, because they are hard to change later.

The log is the rule's output, not diagnostics. In every example in this book, `APPROVE` and `REJECT` record what they decided by logging it. That is the decision trail an auditor reads, and it should go somewhere durable and queryable, not to a rolling text file.

Levels are yours. BUBAS passes the string through untouched. This book uses `DECISION`, `NOTE` and `RECORD` because they are what the domain calls them, and nothing forces `INFO`/`WARN`.

Where the real work goes

The recurring failure in an embedded language is a vocabulary that quietly becomes the application.

A thin handler translates and delegates. It converts BUBAS values into domain calls, calls something that already existed, and converts back. If a handler is more than a few lines, ask what it is doing that a domain service should have been doing.

Two smells worth naming:

A handler that reads several services and combines them. That combination is domain logic living in a translation layer, where it is hard to test and impossible to reuse from anywhere but BUBAS. It belongs in a service.

A handler that branches on the values it was given. A decision inside a handler is a decision that has left the rule — chapter 23's warning about `CHECK_EXPENSE_POLICY`, in miniature. Sometimes it is legitimate (`ANOMALY_SCORE_OF` has to decide something), but it is always worth a second

look.

The test is the same one chapter 23 gave: *does this encode a decision somebody could disagree with?* If yes, it wants to be in the rule or behind a named service, not in the glue.

Configuration

Thresholds, caps and limits are the interesting case, and the answer runs against instinct.

Do not configure them. A cap that lives in a properties file has left the rule, and everything this book argues for has been given up quietly: it is no longer visible to the reviewer, no longer part of the artefact under review, no longer version-controlled with the logic it governs.

Chapter 6's rule declares `ceiling` as a `FINAL` in the program, where a finance manager reads it. That is the right place. If a number varies by tenant or by period, it becomes a **parameter** — which is what `limit` is throughout this book — and the application supplies it per run.

One caveat, and chapter 8 is where it bites. A `FINAL` is fine as a *figure* — a cap compared against a total nobody has worked out yet is a real comparison. It is not fine as a *switch*: a `FINAL` flag read by an `IF` is a test the compiler can answer, and it is refused. Anything the rule's behaviour turns on is a parameter, not a constant.

What genuinely belongs in configuration is infrastructure: endpoints, credentials, timeouts, which model version chapter 25 told you to pin. Nothing a rule-writer would recognise.

The shape it settles into

Most applications end up with the same four pieces, and it is worth aiming at them directly:

- **One place that builds and seals the language**, at startup, once.
- **One place that compiles rules**, whenever the rule text changes, holding the compiled programs.

- **A small factory** that turns a request into a configured interpreter with its services and logger.
- **Handlers that do nothing but translate.**

Everything else is your application, unchanged and unaware that a language is involved. That is the sign it is wired correctly: the vocabulary is a thin edge on a system that would still make sense without it.

CHAPTER 29

Concurrency and lifecycle

Seal once, compile once, interpret per request. What is shared safely between threads, what must never be, and how this maps onto a web application, a batch job, and a queue consumer.

The whole rule

Object	Made	Shared between threads
`BubasLanguage`	once, at startup	yes, freely
`BubasProgram`	once per rule text	yes, freely
`Interpreter`	once per execution	**never**

That is the entire concurrency story, and it holds because of a decision made much earlier: a BUBAS program has one global scope, no local variables, and no concurrency within itself. An interpreter therefore owns all the state of one run, and two runs cannot see each other because they are two objects.

There is no lock anywhere in the interpreter, because there is nothing to lock.

Sealing at startup

Build the language once, in application startup, and hold it for the life of the process.

This is where chapter 24's overlap analysis runs, so an ambiguous vocabulary is a **startup failure** — the process does not come up, with a message naming both patterns. That is the behaviour you want: better a deployment that fails immediately than one that succeeds and behaves unpredictably on the first program that happens to be ambiguous.

Do not seal lazily, and do not seal per request. It is expensive by design, once.

Compiling when the rule changes

A `BubasProgram` is expensive relative to a run and cheap relative to startup. Compile when the rule text changes and hold the result.

For rules that ship with the application, compile at startup beside the sealing. For rules that people edit at runtime — chapter 31's subject — compile on save, keep the compiled program in a map keyed by whatever identifies the rule, and replace the entry when the text changes.

Replacing an entry is safe without coordination. A `BubasProgram` is immutable, so a thread that picked up the old one mid-request finishes with the old one, and the next request gets the new one. There is no window in which a half-updated rule can run. A plain `volatile` field or a `ConcurrentHashMap` is sufficient; nothing here needs a lock.

Interpreting per run

One interpreter per execution, used once, discarded. It is a small object and allocating one is not a cost worth optimising.

The temptation to pool them should be resisted. An interpreter holds the variables of a run; a pooled one holds the variables of somebody else's claim, and the bug that follows is the worst kind — intermittent, data-dependent, and invisible in testing.

Three shapes

A web application. Seal at startup. Compile rules at startup or on change. Per request: build an interpreter, register the request-scoped services, supply the arguments, run, use the answer. The interpreter's lifetime is inside the request, which makes it the natural place for a transaction or a tenant-specific store.

A batch job. Seal once, compile once, then a fresh interpreter per record. Parallelising across records needs nothing beyond not sharing an interpreter — the language and the program are already safe, and there is no shared mutable state to contend on. This is where the design pays off most visibly: throughput scales with cores, and there is no synchronisation to reason about.

A queue consumer. The same as a batch job, with the added case of redelivery. A rule that has run once and is run again on a redelivered message

will do its side effects twice, because BUBAS has no notion of a transaction. Idempotency is the application's problem, and the place to solve it is in the handlers or around the consumer — not in the rule, which should not know that queues exist.

What has an answer, and what has not

A run can be bounded, and by default is not. `maxSteps` on the interpreter counts statements executed and loop passes taken, so a rule that loops forever stops at a number you chose rather than never. `maxArrayLength` caps what a single `DECLARE lines[n] Item` may bring into existence, which is the allocation an expression-sized array otherwise makes without anybody agreeing to it. Chapter 28 shows both. They are unlimited unless you set them, so a program that has never been bounded is still a program that can loop forever — the default is the gap, not the mechanism.

There is still no deadline. The budget counts what the program does, and an operation you registered that blocks for an hour is one step. A rule that calls a slow service is slow, and no number stops that. If you need a wall clock, it is your thread and your timeout, outside all of this.

And the memory bound is per array, not per run. Ten thousand small arrays, each inside the cap, are ten thousand arrays. The limit stops the single absurd allocation, which is the mistake that actually happens; it is not a heap quota.

The shape of it: a rule from a colleague who wrote a slow loop is now a rule that stops. A rule written to exhaust you is a different problem, and the last two chapters take it up.

CHAPTER 30

Failing well

Two kinds of failure with two audiences. A compile error is shown to the person writing the rule and should name their line; a runtime error is an operations problem and should never be shown to them at all. What to log, what to surface, and what to do with a program that will not compile.

Two failures, two audiences

Everything that can go wrong falls into one of two boxes, and confusing them is how an application ends up showing a stack trace to somebody in finance.

A rule that will not compile is a problem for whoever wrote the rule. It happens when they save it, before anything runs, and the message is about their line.

A rule that fails while running is a problem for whoever operates the system. It happens on a particular claim, in production, and the rule-writer usually cannot do anything about it.

They arrive as the same Java exception type, which is why it is worth being deliberate.

Compile failures belong to the author

`compile` throws a `BubasException` carrying the line, the source line and a diagnostic:

```
line 14: 'lastMerchant' is read before it is assigned (at 14:24)
        NOTE "last was " + lastMerchant + ", limit " + limit
```

Show that. All of it, unedited, next to the editor the author is typing in.

The temptation is to wrap it — "the rule could not be saved, please contact support" — and it destroys the thing chapter 8 built. That message names a variable, a line, and what is wrong with it, in words a non-programmer can act on. It is the entire feedback loop that lets somebody in finance write a rule at all.

Three things worth doing around it:

Compile on save, not on deploy. A rule that fails to compile should fail in front of the person who wrote it, seconds after they wrote it.

Never store a rule that does not compile. If it cannot be stored, it cannot be deployed by accident, and the invariant "every stored rule compiles" is worth a great deal later.

Do not translate the message. Rewording it into your own vocabulary means maintaining a mapping that will drift, and the originals were written for this audience.

Runtime failures belong to operations

A `BubasException` from `run` means something went wrong on this claim: a division by zero, an index past the end of an array, an overflow, or a handler that threw.

That last one is the common case, and it is worth seeing clearly. When `TOTAL_OF` cannot reach the claim store, the failure surfaces as a run failure of the rule — but the rule is not wrong, and its author cannot fix it. Showing them anything is a mistake.

So: log it with everything needed to diagnose — the rule, the claim, the diagnostic, the cause — and give the caller whatever your application gives when something breaks. The rule-writer hears about it only if the rule really is at fault, and telling them apart is a judgement your application makes, not one BUBAS can make for you.

What to log

The decision trail and the failure trail are different things and want different destinations.

Decisions are what the rule recorded through `ctx.log` — approved, refused, escalated, and why. This is the audit trail. It should be durable, queryable, and joined to the claim, because in a year somebody will ask why this claim was refused and the answer has to exist.

Failures are exceptions. These go wherever your other exceptions go.

Keep them apart. A decision trail with stack traces in it is not a decision trail, and an error monitor full of ordinary refusals is an error monitor nobody reads.

Failing inside a handler

You decide what a handler does when the world does not cooperate, and the choice is not obvious.

Throwing stops the run. The claim is not decided, the caller sees a failure, and nothing partial has happened — except for whatever earlier statements already did, which is the wrinkle. BUBAS has no transactions, so a rule that approved and then failed while notifying has approved.

Returning a conservative value lets the rule continue on an assumption nobody wrote down. A score of 1 when the model is unreachable means every claim looks ordinary during an outage.

There is no general answer, and the harm is in choosing silently. Whichever you pick, say so in the operation's description — chapter 26's document is where a rule-writer will look — and log the substitution when it happens, so that "why did nothing get flagged last Tuesday" has an answer.

The failure that is not a failure

One thing to get right in the plumbing: a rule returning `FALSE` is not an error.

Every rule in this book answers `FALSE` when a claim is not approved. That is a decision, and a correct one. An application that treats a falsy return as a failure — retrying, alerting, or raising — will alert on every refused claim, and the alerts will be turned off within a week.

Obvious written down; less obvious at four in the afternoon when wiring the caller.

CHAPTER 31

Where programs come from

Rules have to be written, stored, versioned, reviewed and deployed by people who are not using your source control. Authoring, storage, review workflow built on the export, and rolling back a rule that turned out to be wrong.

The question the language does not answer

BUBAS compiles a string. Where the string comes from is your problem, and it is the problem that decides whether any of this works in practice.

If rules live in your repository and change through pull requests, you have not gained much — the person who owns the policy still cannot change it without an engineer. That arrangement is legitimate and simpler, and plenty of teams should stop there. But the argument of chapter 1 only fully pays off when the rule-writer can write.

Authoring

The minimum useful editor is a text box, a Save button, and the compiler's message displayed on failure. That is genuinely enough to start, because chapter 30's diagnostics are written for this audience.

What repays effort after that, roughly in order:

The vocabulary document beside the editor. Chapter 11's export, on the same screen. A rule writer's commonest question is "what can I say", and the answer already exists in a form they can read.

Compile on keystroke. The compiler is fast enough, and there is a large difference between finding out on save and finding out as you type.

Never store a rule that does not compile. Chapter 30 made this point; it belongs here as a storage invariant. Every stored rule compiles, so deployment can never fail for that reason.

What does not repay effort is a visual builder. Blocks and dropdowns look friendlier and are worse: they are harder to review, harder to diff, harder to search, and they hide the artefact that the whole design exists to make readable. The reason a rule is eleven readable lines is so that people can read it.

Storage and versioning

Rules are content, and want what content wants: a store, a history, and an identity that survives edits.

Three properties are worth insisting on.

Every version is kept. When a claim was decided in March, the question "what did the rule say then" must have an answer. Overwriting destroys the audit trail that chapter 30's decision log depends on.

Decisions record which version decided them. A rule identifier and a version, stored with the outcome. Without it the decision log says what happened but not why, and the two drift apart the first time a rule changes.

Rolling back is a deploy, not an edit. Activating an earlier version should be one action with its own record — not somebody pasting old text into the editor, which produces a new version that merely resembles the old one.

None of this is exotic; it is what any content system does. It is worth stating because teams reach for a database column and discover the requirements one incident at a time.

Review

This is where the export earns its keep, and where the workflow differs from code review.

Review the vocabulary once, before the rules exist. Chapter 26 argued this and it is the step people skip. A vocabulary agreed at the start is a set of operations somebody with domain authority confirmed the domain has. Reviewed after fifty rules depend on it, review can only ratify.

Review rules the way you review policy, not the way you review code. The reviewer is checking whether the rule says what the policy says. They need

the rule, the policy clause, and the tests — and because of Part 2, they can read all three.

Make the tests part of the review. A rule arriving without cases for its thresholds is a rule whose author has not thought about the boundaries. Chapter 20's list is a reasonable standard to hold people to.

Deployment

The useful property, and the reason to keep rules out of the application artefact: **a rule change is not a deployment**.

A new rule version compiles, its tests run, and it becomes active — without rebuilding or restarting anything. Chapter 29 explains why that is safe: a `BubasProgram` is immutable, so replacing a map entry needs no coordination and no request sees a half-updated rule.

Two things to build in from the start.

Run the rule's tests before activating it. You have a test framework the rule-writer can read; running the suite on save and refusing activation on failure is a small amount of plumbing for a large amount of safety.

Make activation reversible in one action. The previous version is compiled and sitting in the store. Going back should take seconds and leave a record, because the moment you need it you will not want to be reading this chapter.

What good looks like

A rule-writer opens an editor, sees the operations available, writes a rule, sees it compile as they type, writes three cases for its thresholds, sees them pass, and sends it for review. A reviewer reads the rule against the policy, reads the cases, and approves. Activation happens without a deployment, and the decision log from that moment names the version.

Nothing in that requires an engineer, and every step is something a person can be held to. That is the arrangement this book has been building toward, and it is worth noticing that the language was only ever part of it.

CHAPTER 32

Programs written by a model

The generation case, end to end. What a finite vocabulary actually bounds, what it does not — an operation you expose can do anything Java can do — and where the remaining risk sits. The allow-list argument in full, with its limits stated.

The argument, restated where it can be examined

Chapter 1 made a case with nothing behind it yet: do not constrain a large language, supply a small one. Thirty chapters later the small language exists, and the case can be stated precisely.

Ask a model for an expense rule in Java and you have granted the file system, the network, reflection, process execution, and every class on the classpath. Nobody intended to grant any of it; it arrives with the language. The guardrails people reach for — review the diff, run static analysis, forbid imports, sandbox it — are all deny-lists over an unbounded space.

Ask for the same rule in a BUBAS vocabulary and the set of things the output can *name* is the list you wrote. `DELETE_ALL_CLAIMS` is not rejected on policy grounds; it means nothing, and the program does not compile, for the same reason a typo does not compile.

That is the whole mechanism. It is worth being exact about what it does and does not buy.

What generation actually looks like

Four things to get right, and the first two matter most.

Give the model the vocabulary document. Chapter 26's `asJson()` exists for this. A model told what operations exist, what they take and what they answer, produces programs that compile; one left to guess produces plausible names that do not.

Compile before you look. The compiler is the first reviewer and it is free. Anything that does not compile never reaches a person. Chapter 8's list — a value read before it is worked out, a path that does not return, a type mismatch — is caught here without anybody's attention.

Run the tests. If the rule was requested along with cases, they run before a human sees anything. A rule that compiles and fails its own cases is not worth reviewing.

Then have a person read it. Not an engineer — the person who owns the policy. That is the entire point, and it is available because the output is eleven readable lines rather than two hundred of Java.

The loop is worth stating as a shape: **generate, compile, test, review by the owner, activate with a version.** Three of those five are automatic.

What the vocabulary bounds

Precisely this: **the set of operations a generated program can name is finite, written down, and was reviewed by a person before anything was generated.**

Everything else follows from it. A generated rule cannot reach a system nobody exposed. It cannot grow a dependency on a field somebody is about to rename, because there are no fields. It cannot smuggle behaviour past a reviewer in a helper method, because there are no helper methods. It cannot do anything the vocabulary does not have a word for.

And the review that matters becomes possible. A finance manager can read a generated expense rule and say whether it matches the policy — which is not a thing they can do with generated Java, at any level of model competence.

What it does not bound

Four limits. Overstating the mechanism is how the idea gets dismissed, so they are worth as much space as the benefits.

It bounds naming, not doing. An operation you expose can do anything Java can do. `RUN_SHELL_COMMAND` is a perfectly registrable operation. The vocabulary is exactly as narrow as the operations you chose, and choosing badly

gets you the exposure you were avoiding. The mechanism moves the security decision from *reviewing output* to *designing a vocabulary* — earlier, once, and by someone qualified, but it does not remove it.

It does not make the rule correct. A generated rule that compiles, passes its tests and reads plausibly can still encode the wrong policy. That is what the human review is for, and no amount of constraint substitutes for it.

It does not bound resources. Chapter 29 said this and it applies with force here: a generated program can loop forever, and nothing stops it. For untrusted generation this is the gap that matters most, and it needs containment outside the language.

It does not bound what the operations are asked to do. A rule that calls `APPROVE` on every claim is fully within the vocabulary and entirely wrong. Damage does not require exotic capability; it requires the ordinary capability applied wrongly, which is why the review is not optional.

Where the risk actually sits

Ranked by how much they should worry you.

Vocabulary design. This is nearly all of it. An operation exposed carelessly is a permanent grant, and the model will find it. Chapter 23's test — *does this encode a decision somebody could disagree with?* — is the load-bearing one.

Absent resource limits. Real, unsolved, and the reason untrusted generation needs a thread you can watch and abandon.

Volume. A model can produce fifty plausible rules in a minute, and review capacity is a person's afternoon. The bottleneck moves to the review, and a team that responds by reviewing less has given up the mechanism while keeping the paperwork. If you cannot review it, do not generate it.

Model competence. Last, deliberately. It is what people worry about first and it is the concern that expires: it improves on its own, and the compiler catches most of what it gets wrong. The structural limits above do not improve on their own.

When not to generate at all

If nobody can review the output, generating it is worse than not having it, because it produces artefacts that look reviewed.

If the vocabulary has operations nobody has thought hard about, fix that first. Generation multiplies whatever the vocabulary permits.

And if the rules are stable, few, and already understood — generation is solving a problem you do not have. The vocabulary is still worth building, for chapter 1's original reason. The generation is optional.

CHAPTER 33

Running someone else's rules

Everything left: resource limits, what they now bound and what they still do not, auditing the decisions a rule made and why, observability, and what containment untrusted input still needs. Ends with the complete application assembled.

Three degrees of trust

"Someone else's rules" covers three situations that need different things, and conflating them leads either to paranoia or to a bad afternoon.

Your colleagues' rules. People in your organisation, accountable, reviewed. This is what the book has assumed throughout, and nothing in this chapter's harder half applies.

Rules from a model. Chapter 32. Reviewed by a person before activation, so the trust question is really a review-capacity question — with one exception, below.

Rules from outside. A customer writing rules in your product, a tenant configuring their own approval policy. Nobody reviews these. This is the case BUBAS is not yet ready for, and saying so is more useful than implying otherwise.

The gap, stated plainly

Some of what this chapter used to call missing is now there, and the part that remains is the harder part.

A run can be bounded. `maxSteps` stops a program that loops forever; `maxArrayLength` refuses the single enormous `DECLARE lines[n] Item`. Both are per run, both default to off, and a rule that has never been given a budget still runs until it finishes. Chapter 28 shows the two lines.

There is still no deadline. The budget counts steps the program takes, and a call into an operation you registered is one step however long it blocks. An

untrusted rule cannot spin, but it can call `LOAD_CLAIM` in a loop and make your database do the spinning.

The memory bound is per array. It stops one absurd allocation, not a thousand ordinary ones.

And nothing bounds what an operation does. This is the real remainder, and no limit on the interpreter reaches it: the vocabulary is your code, and a rule can only call what you exposed. That is a design property rather than a gap, but it means the budget protects you from the *program* and not from the *vocabulary*.

So an untrusted rule still needs a thread you watch from outside, with a hard decision about what to do when it will not stop — but the decision now arrives for a narrower set of reasons, and a rule that merely loops is no longer one of them. A process boundary is still more honest if the scale justifies it.

The honest summary has moved by one word: **BUBAS bounds what a rule can name, and now bounds some of what it can consume.** For the first two degrees of trust that was already enough. For the third it is closer, and the remaining distance is a deadline and a way to bound the operations themselves.

Auditing decisions

If a rule decided something about a person, that decision will be questioned, and the question arrives long after everyone has forgotten the details.

Five things make the answer available. Store them together, with the outcome:

- **The claim** — what was decided about
- **The rule identifier and version** — chapter 31's requirement, and the one people skip
- **The decision and its stated reason** — what `APPROVE` and `REJECT` logged
- **The inputs the application supplied** — the limit, the flags, whatever varied per run
- **What non-deterministic operations answered** — chapter 25's requirement, and the only way an anomaly score is ever explicable

With those, "why was this refused in March" is a query. Without the version, you have what happened and not why. Without the model's answer, you cannot explain the one decision most likely to be challenged.

A useful property falls out of the design: the reason strings in the decision log are the same sentences a rule-writer wrote and a reviewer read. The explanation you give an auditor is not a reconstruction — it is the artefact.

Observability

Three things worth measuring, and they are not the usual three.

Compile failures per author. Not an error rate — a usability signal. A rule-writer failing repeatedly on the same diagnostic has found either a gap in the vocabulary or a message that does not say what it means. Both are worth knowing.

Which operations are actually called. The vocabulary is finite, so this is a complete list, not a sample. Operations nobody calls are either missing a use case or should be removed; an operation suddenly called ten times more often is a policy change somebody made.

Decision mix over time. Approvals, refusals and escalations as proportions. A rule change that shifts the mix is doing something, and if nobody expected it to, that is the alert worth having. This is far more useful than latency, which is not going to be the problem.

The complete picture

Everything the book has built, in the order it runs:

At startup. Build the vocabulary and seal it. Chapter 24's overlap analysis runs here, so an ambiguous vocabulary fails the deployment rather than a claim. Compile whatever rules ship with the application.

When somebody writes a rule. They see the vocabulary document beside the editor, the compiler answers as they type, and a rule that does not compile is never stored. They write cases for the thresholds; the cases run on save.

When somebody reviews it. The person who owns the policy reads the rule against the policy and reads the cases. Both are in a language they read. Activation is recorded and reversible.

When a claim arrives. A fresh interpreter, the request's services, the arguments, one run. The decision and its reason go to the audit store with the rule's version.

When something goes wrong. A compile failure reaches the author with its line intact. A runtime failure reaches operations and not the author. The two trails stay separate.

When the vocabulary changes. The export is regenerated, the checksum says which descriptions need re-reading, and somebody with domain authority reads them — ideally before the rules that depend on them exist.

What it was all for

A pull request arrives. A few hundred lines of Java implementing the new expense policy. It compiles, the tests pass, and the person who owns the rule cannot read it.

That was chapter 1, and the arrangement in this book is one answer to it. The rule is eleven lines in a language with no vocabulary of its own. The test is in the same language. The list of things the rule can say is a document somebody agreed to before any of it was written. The reason a claim was refused is a sentence the reviewer read.

None of that makes the rule correct. It makes the rule *reviewable by the person who knows whether it is correct*, which is the only thing that was ever missing.